

Energieausbeute von Solarzellen unter Berücksichtigung meteorologischer Daten – Eine Erweiterung für die Simulationssoftware SETFOS

Autor:innen	My-Hanh Dang Grisha Schuhmacher
Hauptbetreuung	Beat Ruhstaller
Nebenbetreuung	Balthasar Blülle
Industriepartner	Fluxim AG
Datum	19.12.2025

Zusammenfassung

Diese Arbeit befasst sich mit der Erweiterung des Simulationswerkzeugs SETFOS zur Berücksichtigung meteorologischer Einflüsse auf die Energieausbeute von Solarzellen. Aufbauend auf den bestehenden physikalischen Modellen wurde eine Schnittstelle entwickelt, die Datensätze Typischer Meteorologischer Jahre einbindet und damit realitätsnahe Umweltbedingungen in die Leistungsberechnung integriert. Neben strukturellen Anpassungen im Simulationskern wurden Funktionen zur Verwaltung dieser Datensätze in der Benutzeroberfläche ergänzt. Durch diese Erweiterung wird eine präzisere Prognose des Energieertrags ermöglicht und die Anwendbarkeit des Werkzeugs im Forschungskontext deutlich erhöht.

Vorwort

Die Entwicklung moderner Solarzellentechnologien erfordert Werkzeuge, die den realen Betrieb möglichst genau abbilden. Während viele Simulationsumgebungen den Zellaufbau detailliert erfassen, bleiben reale meteorologische Einflüsse oft unberücksichtigt oder werden durch Formeln abstrahiert. Die vorliegende Arbeit entstand im Interesse einer praxisnäheren Modellierung und basiert auf der Zusammenarbeit mit Anwendern und Entwicklern von SETFOS. Unser Dank gilt allen, die durch fachliche Diskussionen und technische Unterstützung zur Umsetzung des Projekts beigetragen haben.

Inhaltsverzeichnis

Zusammenfassung	2
Vorwort	2
Inhaltsverzeichnis	3
1 Einleitung	1
1.1 Ausganglage	1
1.2 Zielsetzung	1
2 Theoretische Grundlage	1
2.1 Solarenergie	1
2.1.1 Was sind Solarzellen und Solarmodule?	1
2.1.2 Wie funktioniert die Stromerzeugung?	2
2.1.3 Einflüsse in der realen Solarzellen	2
2.2 Typical Meteorological Years und PVGIS	3
2.3 TMY-Kopplung als Datengrundlage in der PV-Leistungsmodellierung	3
2.4 Erforderliche Eingabeparameter für den PV-Leistungsmode	4
2.5 Berechnung und Weiterverarbeitung der TMY-Daten im PV-Leistungsmodell	5
2.6 Validierung und statistische Metriken	5
3 Vorgehen & Methoden	6
3.1 GUI	6
3.2 Kernel	7
3.3 Systemablauf und Implementierungsworkflow	7
4 GUI-Code-Implementierung	10
4.1 Zentrale Standort-, Zeit- und TMY-Verwaltung in der LocationAndTimePanel Klasse	11
4.1.1 Aufgaben und Verantwortlichkeiten	11
4.1.2 Architektur und technische Umsetzung	11
4.1.3 Ablauf und wichtige Funktionen	11
4.2 Datenstruktur TMY in der TMYData Klasse darstellen	12
4.2.1 Aufgaben und Verantwortlichkeiten	13
4.2.2 Architektur und technische Umsetzung	13
4.2.3 Ablauf und wichtige Funktionen	13
4.3 Konfiguration und Analyse der TMYs in der TMYSettingsPanel Klasse	13
4.3.1 Aufgaben und Verantwortlichkeiten	13
4.3.2 Architektur und technische Umsetzung	13
4.3.3 Ablauf und wichtige Funktionen	14
5 Kernel-Code-Implementation	15
5.1 Erweiterung der ParserBase.cpp zur Unterstützung von TMY-Parametern	16
5.1.1 Aufgaben und Verantwortlichkeiten	16
5.1.2 Architektur und technische Umsetzung	16
5.1.3 Ablauf und wichtige Funktionen	17
5.2 Erweiterung des ParParser.cpp zur Verarbeitung von TMY-Konfigurationsparametern	17
5.2.1 Aufgaben und Verantwortlichkeiten	17

5.2.2	Architektur und technische Umsetzung.....	18
5.2.3	Ablauf und wichtige Funktionen.....	18
5.3	Erweiterung von MultiLayerOpticsParameter.h zur Steuerung der TMY-Nutzung.....	18
5.3.1	Aufgaben und Verantwortlichkeiten	19
5.3.2	Architektur und technische Umsetzung.....	19
5.3.3	Ablauf und wichtige Funktionen.....	19
5.4	Erweiterung der TaskIlluminationParameter.h zur Steuerung der TMY-Nutzung.....	20
5.4.1	Aufgaben und Verantwortlichkeiten	20
5.4.2	Architektur und technische Umsetzung.....	20
5.4.3	Ablauf und wichtige Funktionen.....	20
5.5	Erweiterung von GenericTaskParameterSetters.cpp und .h zur einheitlichen Initialisierung der TMY-Nutzung	21
5.5.1	Aufgaben und Verantwortlichkeiten	21
5.5.2	Architektur und technische Umsetzung.....	21
5.5.3	Ablauf und wichtige Funktionen.....	22
5.6	Einführung des TMYFileParser zur Verarbeitung meteorologischer Zeitreihendaten.....	22
5.6.1	Aufgaben und Verantwortlichkeiten	22
5.6.2	Architektur und technische Umsetzung.....	22
5.6.3	Ablauf und wichtige Funktionen.....	23
5.7	Erweiterung von EnergyYield.cpp zur TMY-basierten Energieertragsberechnung.....	23
5.7.1	Aufgaben und Verantwortlichkeiten	23
5.7.2	Architektur und technische Umsetzung.....	23
5.7.3	Ablauf und wichtige Funktionen.....	23
6	Resultate	24
6.1	GUI	24
6.1.1	Neuer Workflow zur Verwendung von TMY-Daten	24
6.1.2	Asynchroner Download mit Fehlerbehandlung und Debounce-Mechanismus.....	25
6.1.3	TMY-Einstellungsfenster zur Visualisierung und Auswahl von Parametern	25
6.1.4	Visualisierung der TMY-Daten.....	26
6.1.5	Integration in die Session-Verwaltung.....	26
6.1.6	Erweiterung der Spektrums-Vorschau	26
6.1.7	Zusammenfassung der Resultate	26
6.2	Kernel	26
6.2.1	Neuer Kernel-Workflow ffür TMY-Daten	26
6.2.2	Erweiterung der Parsing- und Parameterinfrastruktur	26
6.2.3	Zentrale Verarbeitung meteorologischer Zeitreihendaten	27
6.2.4	TMY-basierte Energieertragsberechnung.....	27
1.2.5	Zusammenfassung der Kernel-Resultate	27
7	Diskussion und Ausblick	27

7.1	Technische Bewertung der Umsetzung	27
7.2	Nutzerfreundlichkeit und Bedienbarkeit	27
7.3	Projektmanagement und gewonnene Erkenntnisse.....	28
7.3.1	Aufwand für GUI- und Kernel-Integration.....	28
7.3.2	Planung und Arbeitsstruktur	28
7.4	Ausblick	29
8	Verzeichnisse	30
A.	Literaturverzeichnis	30
B.	Abbildungsverzeichnis	30
•	Anhang	31
A.	Anhang 1: Projektauftrag vom Complexis	31

1 Einleitung

Für die Bewertung neuartiger Zellstrukturen sind daher Simulationswerkzeuge notwendig, die sowohl material- als auch umweltbedingte Faktoren realitätsnah erfassen. SETFOS bildet die physikalischen Prozesse innerhalb der Solarzelle bereits detailliert ab, jedoch sind bisher ort- und zeitabhängige Einflüsse nur statisch modelliert. Um die Aussagekraft der Simulation zu erhöhen, wird in dieser Arbeit eine Erweiterung vorgestellt, die typische Umweltparameter aus realen Messungen in das Modell integriert und damit eine präzisere Energieertragsanalyse ermöglicht.

1.1 Ausgangslage

Die erwartete Energieausbeute von Solarzellen hängt von verschiedenen Faktoren ab. Einerseits vom Zellschichtaufbau und dem Wirkungsgrad, andererseits von dem tageszeit- und ortsabhängigen Sonnenspektrum und den Wetterbedingungen. Die ZHAW Spin-off Firma Fluxim hat mit der Software SETFOS bereits eine Simulationssoftware entwickelt, welche diese Punkte berücksichtigt und weltweit in Forschungslaboren eingesetzt wird [1].

Aktuell fehlt dieser Software noch die Verwendung realer meteorologischer Daten, so dass Aussentemperatur, Wind und Bewölkung mit einbezogen werden und Hersteller von Solarpanels ihre Zellen unter realen Bedingungen simulieren können.

1.2 Zielsetzung

Die bereits existierende Software SETFOS (momentan in Version 6.0) soll insofern erweitert werden, dass Typical Meteorological Year (TMY) Daten des ausgewählten Simulationsortes verwendet werden, um die Effizienz und Energieausbeute der Solarpanels zu berechnen. Diese Daten müssen automatisch über eine API abgefragt werden und die Verwaltung dieser Daten soll keinen Zusatzaufwand für die Benutzer verursachen.

Diese Zielsetzung erfordert Anpassungen und Erweiterungen in beiden Kernteilen der Applikation, im Graphical User Interface (GUI) und im Kernel (zuständig für die Berechnung). Das GUI sei zuständig für das Abrufen und Verwalten der Daten, während der Kernel mit diesen Daten arbeiten muss. Dabei müssen die Berechnungen und Resultate den tatsächlichen physischen Formeln entsprechen und sollten realen Testergebnissen ähnlich bleiben.

2 Theoretische Grundlage

2.1 Solarenergie

Weil die Solarenergie ein wichtiger Bestandteil der Energiewende ist und eine zunehmend zentrale Rolle in unserer Energieversorgung übernimmt,[2][3] ist es essentiell, dass zukünftige Solarpanels mit maximaler Effizienz entwickelt werden können.

Auch besteht ein zunehmender Konkurrenzdruck bei Herstellen von Solarpanels, was zusammen mit anderen Entwicklungen die Preise in die Tiefe treibt [4].

2.1.1 Was sind Solarzellen und Solarmodule?

Der zentrale Bestandteil jeder Photovoltaikanlage ist die Solarzelle, wie in Abb. 1 dargestellt. Sie besteht in den meisten Fällen aus Silizium, einem Halbleitermaterial, das auch in elektronischen Bauteilen wie

Dioden, Transistoren und Computerchips Verwendung findet. Durch die gezielte Dotierung mit Fremdatomen wird ein pn-Übergang erzeugt, der innerhalb des Kristalls ein elektrisches Feld aufbaut. Trifft Licht auf die Solarzelle, werden Ladungsträger aus ihren Bindungen im Kristallgitter gelöst und durch das elektrische Feld zu den äußeren Kontakten transportiert. An diesen Kontakten entsteht dadurch eine elektrische Spannung von etwa 0,5 Volt. Die erzeugte Stromstärke ist abhängig von der Sonneneinstrahlung sowie der Fläche der Solarzelle und bewegt sich zwischen 0 und 10 Ampere.

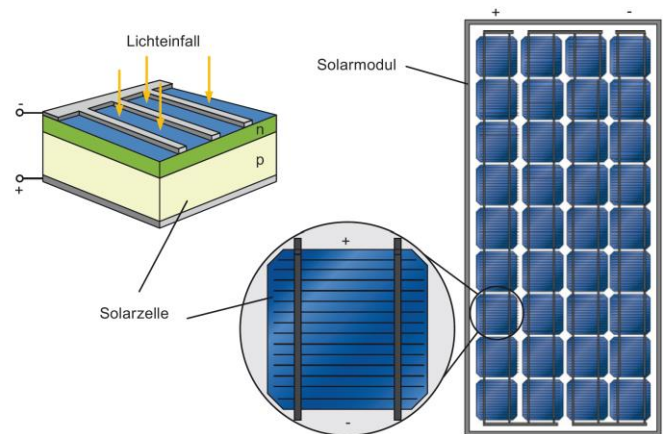


Abb. 1: Solarzelle und Solarmodule als Grundbausteine der Photovoltaik (K. Mertens, 2022, S.32)

Um höhere, technisch nutzbare Spannungen im Bereich von etwa 20 bis 50 Volt zu erreichen, werden mehrere Solarzellen innerhalb eines Solarmoduls in Reihe geschaltet, was ebenfalls in Bild 1.9 veranschaulicht ist. Darüber hinaus erfüllen Solarmodule eine Schutzfunktion, indem sie die Solarzellen mechanisch stabilisieren und durch eine Versiegelung vor Umwelteinflüssen wie dem Eindringen von Feuchtigkeit schützen. Die beschriebenen Zusammenhänge basieren auf der Darstellung in Quelle [5, S. 32].

2.1.2 Wie funktioniert die Stromerzeugung?

Die Solarzelle besitzt – ähnlich wie eine Photodiode – einen pn-Übergang, der asymmetrisch dotiert ist. Dabei befindet sich im unteren Bereich die p-dotierte Basis, während sich auf der Oberseite ein stark n-dotierter Emitter (n^+ -Emitter) befindet. Die Begriffe Basis und Emitter stammen ursprünglich aus der Bipolartransistortechnik und wurden auf die Solarzellentechnologie übertragen (siehe Abb. 2). Trifft Licht auf die Solarzelle, erzeugt jedes absorbierte Photon ein Elektron-Loch-Paar.

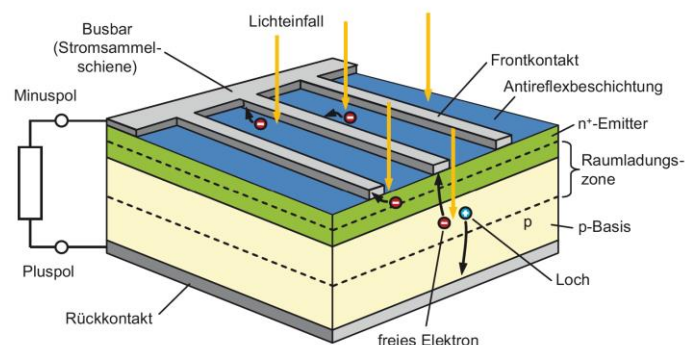


Abb. 2: Typische Silizium-Solarzelle (K. Mertens, 2022, S.92)

Das elektrische Feld in der Raumladungszone sorgt dafür, dass diese Ladungsträger voneinander getrennt und zu den jeweiligen Kontakten transportiert werden: Die Löcher wandern durch die p-dotierte Basis zum rückseitigen Kontakt, während die Elektronen über den Emitter zu den Frontkontakten geleitet werden. Diese Frontkontakte bestehen aus schmalen Metallstreifen, die die Elektronen aufnehmen und über Sammelschienen (Busbars) abführen. Wird ein elektrischer Verbraucher an die beiden Anschlüsse der Solarzelle angeschlossen, kann die erzeugte elektrische Energie genutzt werden (siehe Abb. 2, [5, S. 92]).

Wenn du möchtest, kann ich den Text auch stärker umformulieren, vereinfachen oder an den Stil einer Abschlussarbeit anpassen.

2.1.3 Einflüsse in der realen Solarzellen

In einer realen Solarzelle kommen vielfältige Umwelteinflüsse zusammen, die die Leistungsfähigkeit deutlich verändern können. Dabei spielen sowohl optische als auch elektrische Effekte eine wichtige Rolle.

Zu den optischen Einflüssen zählt unter anderem die Reflexion des einfallenden Sonnenlichts an der Grenzfläche zwischen Luft und Silizium. Aufgrund des deutlichen Unterschieds in den Brechungsindizes ergibt sich dabei eine Reflexion von etwa 35% [5, S. 115]. Ein weiterer relevanter Faktor ist die Abschattung durch die metallischen Kontaktfinger, welche den erzeugten Strom zu den Anschlussdrähten ableiten, jedoch gleichzeitig einen Teil der aktiven Fläche verdecken [5, S. 116]. Zusätzlich können je nach Oberflächentextur der Solarzellen Durchstrahlungsverluste auftreten, wenn Licht nicht hinreichend innerhalb der Struktur eingekoppelt oder mehrfach gestreut wird [5, S. 118].

Zu den elektrischen Einflüssen in realen Solarzellen zählen vor allem Verluste, die durch den Ladungstransport innerhalb der Materialien und an Kontaktstellen entstehen. Ohmsche Widerstände in den metallischen Kontaktstrukturen und im dotierten Halbleiter führen zu Spannungs- und Stromverlusten. Zudem treten an den Übergängen zwischen Metall und Halbleiter potenzielle Barrieren auf, die den Ladungstransfer behindern können, sofern sie nicht durch geeignete Dotierungsprofile minimiert werden [5, S. 118]. Ein weiterer wichtiger Faktor sind rekombinatorische Verluste, sowohl im Volumen des Halbleiters als auch an Oberflächen und Grenzflächen, an denen ungeladene Bindungen Ladungsträger einfangen. Diese elektrischen Effekte beeinflussen gemeinsam die erreichbare Spannung, den Strom und damit die Gesamtleistung der Solarzelle [5, S. 119].

2.2 Typical Meteorological Years und PVGIS

Typical Meteorological Years (TMY) sind Datensätze, die aus einzelnen Monaten unterschiedlicher Jahre zusammengesetzt werden, um ein repräsentatives durchschnittliches Wetterjahr für einen bestimmten Standort abzubilden. Die zugrunde liegenden Daten stammen aus verschiedenen Quellen, unter anderem aus satellitengestützten Strahlungsmessungen. Ein TMY-Datensatz umfasst unter anderem folgende meteorologische und strahlungsbezogene Größen:

- Lufttemperatur
- Relative Luftfeuchtigkeit
- Globale horizontale Strahlungsintensität
- Normale direkte Strahlungsintensität
- Diffuse horizontale Strahlungsintensität
- Infrarotstrahlung
- Windgeschwindigkeit
- Windrichtung
- Luftdruck

Die zeitliche Auflösung der Datensätze beträgt eine Stunde. Die räumliche Auflösung variiert je nach Region und liegt typischerweise zwischen 5×5 km und 25×25 km [6], [7].

Zur Bereitstellung und Nutzung dieser Daten kann das Photovoltaic Geographical Information System (PVGIS) der Europäischen Union herangezogen werden. PVGIS stellt weltweit Informationen zur Sonneneinstrahlung sowie zur Leistungsfähigkeit von Photovoltaiksystemen bereit und ist sowohl über eine grafische Benutzeroberfläche als auch über eine REST-API nutzbar. Über diese Programmierschnittstelle lassen sich TMY-Daten automatisiert abrufen [7].

Mit der Version PVGIS 5.3 wurde das System um die Möglichkeit erweitert, auch Daten für küstennahe Gewässer bereitzustellen. Dadurch können Simulationen an maritimen Standorten durchgeführt werden, was den Anwendungsbereich und die Flexibilität von PVGIS weiter erhöht [8].

2.3 TMY-Kopplung als Datengrundlage in der PV-Leistungsmodellierung

Zur Abschätzung der langfristigen thermischen Betriebsbedingungen eines Photovoltaikmoduls ist die Kenntnis repräsentativer meteorologischer Randbedingungen erforderlich. Da gemessene Wetterdaten

nicht immer verfügbar sind, werden im Paper typische meteorologische Jahresdatensätze (TMY) als alternative Datengrundlage herangezogen. Diese liefern zeitaufgelöste Informationen zur solaren Einstrahlung, Umgebungstemperatur und Windgeschwindigkeit und werden zur Modellierung der PV-Modultemperatur eingesetzt [9].

Die Kopplung des PV-Modultemperaturmodells mit TMY-Daten ermöglicht die Bewertung der Modellgenauigkeit bei Verwendung geschätzter Wetterdaten im Vergleich zu Messdaten und stellt damit einen zentralen Bestandteil der im Paper durchgeführten Analyse dar.

2.4 Erforderliche Eingabeparameter für den PV-Leistungsmod

Die im Paper betrachteten PV-Modelle basieren auf der Berechnung der Modultemperatur, da diese einen wesentlichen Einfluss auf die elektrische Leistungsfähigkeit des Moduls besitzt. Die Leistungsbewertung erfolgt indirekt über den temperaturabhängigen Modulwirkungsgrad.

Die thermische Energiebilanz des PV-Moduls (siehe Abb. 3) wird wie folgt beschrieben [9]:

$$\tau\alpha I_T = \eta_p + U_{loss}(T_c - T_a)$$

Abb. 3: Formel thermische Energiebilanz (N. Aoun, 2022, S.5)

Dabei bezeichnet I_T die solare Einstrahlung auf der Modulfläche, T_c die Zell- bzw. Modultemperatur und T_a die Umgebungstemperatur.

Der Ausdruck $\tau\alpha$ beschreibt das Transmissions-Absorptions-Produkt der Modulabdeckung. Der temperaturabhängige Modulwirkungsgrad (siehe Abb. 4) wird definiert als [9]:

$$\eta_p = \eta_{ref} \cdot [1 - \alpha_p \cdot (T_c - 25)]$$

Abb. 4: Formel der temperaturabhängige Modulwirkungsgrad (N. Aoun, 2022, S.5)

wobei

η_{ref} der Modulwirkungsgrad unter Standardtestbedingungen (STC) und α_p der Temperaturkoeffizient der maximalen Leistung ist.

Die gesamten Wärmeverluste U_{loss} (siehe Abb. 5) setzen sich aus Strahlungs- und Konvektionsverlusten zusammen und werden im Paper wie folgt angegeben [9]:

$$U_{loss} = \varepsilon_{glass} \sigma (T_c^2 + T_{sky}^2) (T_c + T_{sky}) + \varepsilon_{back} \sigma (T_c^2 + T_{gr}^2) (T_c + T_{gr}) + (2.8 + 3W_s)$$

Abb. 5: Formel gesamter Wärmeverluste (N. Aoun, 2022, S.5)

mit der Windgeschwindigkeit v_w , der Stefan-Boltzmann-Konstanten

$\sigma = 5.6697 \times 10^{-8} \text{ W m}^{-2} \text{ K}^{-4}$ sowie den Emissionskoeffizienten der Glas- und Rückseitenabdeckung ε_{glass} und ε_{back} .

Die Boden- und Himmelstemperaturen (siehe Abb. 6) werden gemäß dem Paper berechnet durch [9]:

$$T_{gr} = 17.898 + 0.951T_a$$

$$T_{sky} = 0.0552T_a^{1.5}$$

Abb. 6: Boden- und Himmelstemperaturen
(N. Aoun, 2022, S.5)

Damit ergeben sich als erforderliche Eingabeparameter für das im Paper verwendete PV-Leistungs- und Temperaturmodell:

- solare Einstrahlung auf der Modulfläche I_T ,
- Umgebungstemperatur T_a ,
- Windgeschwindigkeit v_w ,
- modulspezifische Parameter $\eta_{ref}, \alpha_p, \tau\alpha$,
- materialabhängige Emissionskoeffizienten $\varepsilon_{glass}, \varepsilon_{back}$.

Diese Parameter bilden die vollständige Eingangsgrößenbasis der im Paper untersuchten Modelle [9].

2.5 Berechnung und Weiterverarbeitung der TMY-Daten im PV-Leistungsmodell

Im Paper werden TMY-Daten genutzt, um meteorologische Eingangsgrößen bereitzustellen, wenn Messdaten nicht verfügbar sind. Die relevanten Größen sind die solare Einstrahlung, die Umgebungstemperatur sowie die Windgeschwindigkeit. Diese Daten werden zeitschrittweise in die Modultemperaturmodelle eingespeist.

Die berechneten Modultemperaturen dienen ausschließlich dem Vergleich zwischen modellierten und gemessenen Temperaturen. Ziel ist es, den Einfluss geschätzter meteorologischer Daten auf die Genauigkeit der Modultemperaturmodelle zu bewerten. Eine explizite elektrische Leistungs- oder Energieertragsberechnung wird im Paper nicht durchgeführt.

2.6 Validierung und statistische Metriken

Zur Bewertung der Modellgüte werden im Paper statistische Fehlerkennzahlen verwendet. Der Vergleich erfolgt zwischen berechneter Modultemperatur $T_{c,e}$ und gemessener Modultemperatur $T_{c,m}$.

Der Mean Bias Error (MBE) (siehe Abb. 7) ist definiert als [9]:

$$MBE = \frac{1}{N} \sum_1^N (T_{c,e} - T_{c,m})$$

Abb. 7: Formel Mean Bias Error (N. Aoun, 2022, S.5)

Der Mean Percentage Error (MPE) (siehe Abb. 8) lautet [9]:

$$MPE = \frac{100}{N} \sum_1^N \left(\frac{T_{c,e} - T_{c,m}}{T_{c,m}} \right)$$

Abb. 8: Formel Mean Percentage Error
(N. Aoun, 2022, S.5)

Der Root Mean Square Error (RMSE) (siehe Abb. 9) wird berechnet durch [9]:

$$RMSE = \sqrt{\frac{1}{N} \sum_1^N (T_{c,e} - T_{c,m})^2}$$

Abb. 9: Formel Root Mean Square Error (N. Aoun, 2022, S.6)

Der relative RMSE (rRMSE) (siehe Abb. 10) ergibt sich zu [9]:

$$rRMSE = \frac{\sqrt{\frac{1}{N} \sum_1^N (T_{c,e} - T_{c,m})^2}}{\frac{1}{N} \sum_1^N T_{c,m}}$$

Abb. 10: Formel relative RMSE (N. Aoun, 2022, S.6)

Zur Gesamtbewertung der Modelle wird zusätzlich der Global Performance Index (GPI) verwendet [9].

3 Vorgehen & Methoden

Da die Software SETFOS aus zwei verschiedenen Teilen ohne gemeinsame Elemente besteht (namentlich dem GUI und dem Kernel), wurde der Entwicklungsprozess ebenfalls in zwei Teile aufgeteilt. Beide dieser Teile verliefen ungefähr gleich. Zusätzlich wurde am Anfang des Prozesses die grundlegende Theorie studiert, welche in den vorherigen Abschnitten zu finden ist.

Dabei wurden relevante fachliche Grundlagen untersucht, insbesondere meteorologische Datensätze in Form von *Typical Meteorological Year (TMY)*-Daten sowie die *PVGIS-API-Schnittstelle*. Ziel dieser Recherche war es, ein grundlegendes Verständnis der physikalischen Grundlagen von Solarmodulen, der Bedeutung von TMY-Daten für Simulationszwecke sowie deren Bereitstellung und Nutzung über eine API zu erlangen. Zusätzlich wurde analysiert, welche spezifischen Informationen aus den TMY-Datensätzen für die Simulation innerhalb von SETFOS relevant sind.

Auf Basis des gewonnenen fachlichen und technischen Verständnisses wurde anschließend die Projektorganisation strukturiert. Die identifizierten Anforderungen und Aufgaben wurden definiert und mit Hilfe des Projektmanagement-Tools Jira erfasst. Die Hauptaufgaben wurden in mehrere Unteraufgaben unterteilt, wodurch eine grobe zeitliche Planung des Projekts möglich wurde. Regelmäßige wöchentliche Treffen mit den Projektverantwortlichen dienten der Abstimmung des Projektfortschritts, der Klärung offener Fragen sowie der Anpassung der weiteren Vorgehensweise. Die Ergebnisse dieser Besprechungen wurden in Protokollen dokumentiert.

Für beide Teile der Implementierung wurde jeweils ein separater Branch erstellt.

3.1 GUI

Zu Beginn der Arbeit stand das Aufsetzen der Software *SETFOS* im Vordergrund. Ziel war es, eine lauffähige Entwicklungsumgebung zu schaffen, um darauf aufbauend Erweiterungen implementieren zu

können. Gemäss Plan wurde dies zunächst nur für den GUI-Teil der Applikation durchgeführt, wobei eine bereits existierende, vorkompilierte Version des Kernels lokal angehängt wurde. Hierfür wurden zunächst alle notwendigen Bibliotheken und Abhängigkeiten identifiziert und mithilfe des Build-Tools *Maven* installiert, wobei der Maven-Server nur lokal aus dem Netzwerk der Fluxim erreichbar war. Nach erfolgreicher Installation wurde überprüft, ob die Software fehlerfrei startet und die bestehenden Funktionalitäten ordnungsgemäss ausgeführt werden. Aufgrund der speziellen Infrastruktur und teils unklaren Dokumentation, sowohl der nur teilzeitigen Verfügbarkeit des Projektteams dauerte dieser Schritt eine ganze Woche.

Im nächsten Schritt erfolgten eine umfassende Recherche und Codeanalyse. Dazu wurde die bestehende Architektur von SETFOS untersucht, um geeignete Stellen für die Integration der neuen Funktionalitäten zu identifizieren.

Die Code-Implementierung erfolgte anschließend. Im GUI-Teil wurden die notwendigen Erweiterungen der grafischen Benutzeroberfläche umgesetzt, um dem Nutzer die Auswahl, Konfiguration und Anzeige der TMY-Daten zu ermöglichen. Dabei wurde darauf geachtet, dass sich die neuen Eingabe- und Anzeigeelemente nahtlos in die bestehende Benutzeroberfläche von SETFOS integrieren.

3.2 Kernel

Wie auch beim ersten Teil musste der Kernel zunächst lokal auf den Systemen des Entwicklungsteams aufgesetzt werden.

Der Kernel-Teil umfasste die Implementierung der eigentlichen Programmlogik. Hier wurden die Funktionen zur Anbindung an die PVGIS-API, zum Abruf und zur Verarbeitung der TMY-Daten sowie zur Einbindung dieser Daten in die Simulationsabläufe von SETFOS realisiert. Die neuen Funktionen wurden so implementiert, dass sie mit der bestehenden Softwarearchitektur kompatibel sind und eine Erweiterbarkeit für zukünftige Anpassungen ermöglichen.

3.3 Systemablauf und Implementierungsworkflow

Um die Integration der TMY-Daten und der PVGIS-API in SETFOS nachvollziehen, wird im Folgenden der gesamte Ablauf von der Benutzereingabe bis zur Ergebnisdarstellung beschrieben. Der dargestellte Workflow verdeutlicht das Zusammenspiel zwischen grafischer Benutzeroberfläche (GUI) und Kernel und zeigt, wie externe meteorologische Daten in die Simulationsumgebung eingebunden werden. Der Gesamtprozess ist schematisch in Abb. 3 dargestellt.

1. GUI-seitige Prozesse

1. Benutzereingabe und Parameterauswahl

Über die GUI gibt der Nutzer standortbezogene Parameter wie geografische Koordinaten und den Simulationszeitraum ein. Neu kann der Benutzer auch die Verwendung von TMY-Daten aktivieren. Nicht erwähnt ist dabei das Einstellen der Solarzellenstruktur.

2. Validierung der Eingaben

Die eingegebenen Parameter werden auf Vollständigkeit und Plausibilität geprüft, um fehlerhafte Eingaben frühzeitig zu vermeiden.

3. Anfrage an die PVGIS-API

Sofern die Verwendung von TMY-Daten aktiviert ist, erzeugt die GUI eine Anfrage an die PVGIS-API zum Abruf der standortbezogenen TMY-Daten.

4. Abruf der meteorologischen TMY-Daten

Die PVGIS-API liefert die angeforderten TMY-Daten zurück, welche unter anderem Informationen zur Globalstrahlung, Temperatur und Windgeschwindigkeit enthalten. Sollte dabei ein Fehler auftreten, so wird dies dem Nutzer mitgeteilt. Die Daten werden im JSON-Format gespeichert.

5. Übergabe der Parameter an den Kernel

Sobald der Benutzer die Ausführung der Simulation anordnet, werden alle Daten in strukturierter Form an den Kernel weitergeleitet und dienen dort als Grundlage für die weitere Verarbeitung.

2. Kernel-seitige Prozesse

6. Datenverarbeitung und Validierung

Die empfangenen Daten werden verarbeitet, auf Konsistenz geprüft und in das interne Datenformat von SETFOS überführt. Fehlerhafte oder unvollständige Datensätze werden entsprechend behandelt.

7. Durchführung der Simulation

SETFOS führt die Simulation unter Berücksichtigung der meteorologischen Daten und der definierten Systemparameter durch und berechnet die relevanten Ergebnisgrößen.

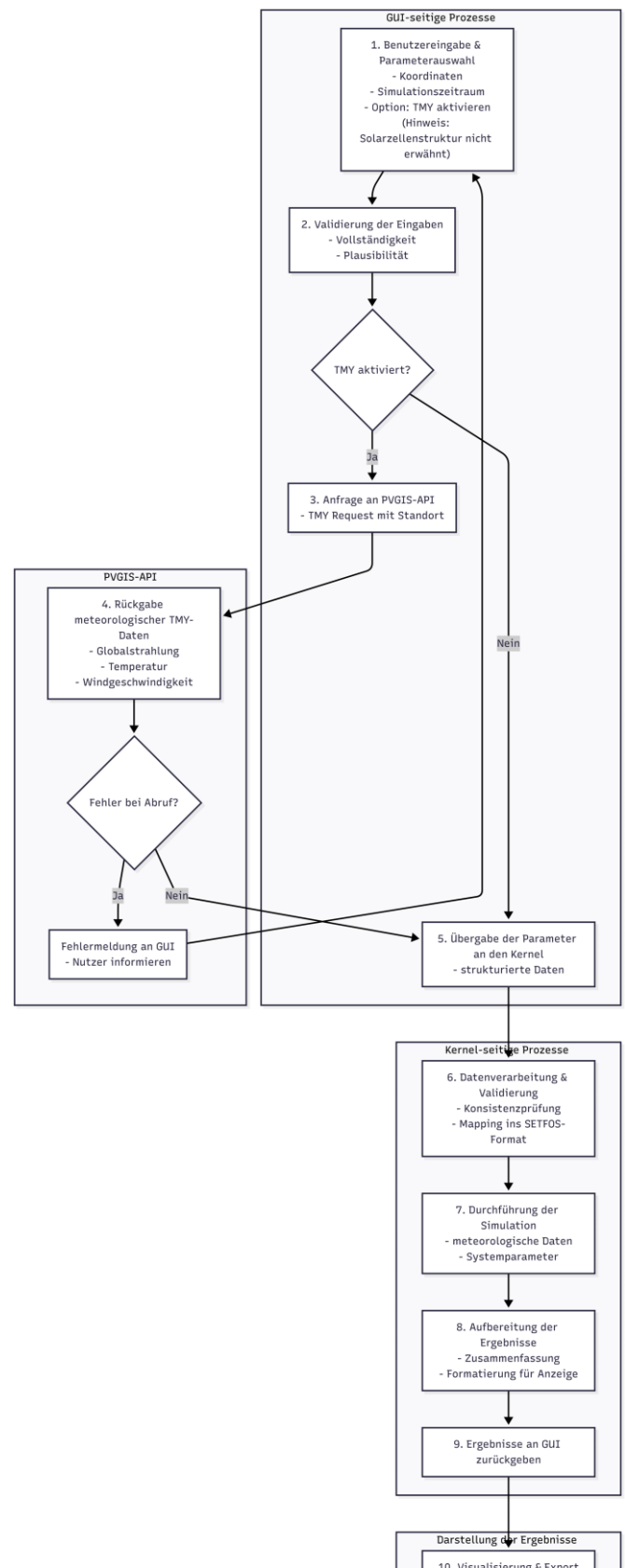


Abb. 11: Systemablauf und Implementierungsworkflow (eigene Abbildung)

8. **Aufbereitung der Simulationsergebnisse**
Die Simulationsergebnisse werden zusammengefasst, formatiert und für die Darstellung vorbereitet.
9. **Übertragung der Ergebnisse an die GUI**
Die aufbereiteten Simulationsergebnisse werden vom Kernel an die GUI zurückgegeben.
3. **Darstellung der Ergebnisse (GUI)**
 10. **Visualisierung und Export**
Die Ergebnisse werden dem Nutzer in geeigneter Form dargestellt. Optional können diese gespeichert oder exportiert werden.

4 GUI-Code-Implementierung

Die grafische Benutzerschnittstelle wurde in *Java Swing* realisiert. Zur Erweiterung des bestehenden Funktionsumfangs wurden die vorhandene Klasse *LocationAndTimePanel* angepasst sowie die beiden neuen Klassen *TMYData* und *TMYSettingPanel* ergänzt. Zusätzlich mussten verschiedene Definitionen und Übermittlungsfunktionen leicht angepasst werden.

Die Implementierung der grafischen Benutzerschnittstelle folgt einem klar strukturierten Workflow, der die Eingabe standort- und zeitabhängiger Parameter, die optionale Integration meteorologischer TMY-Daten sowie die Initialisierung der spektralen Simulation umfasst.

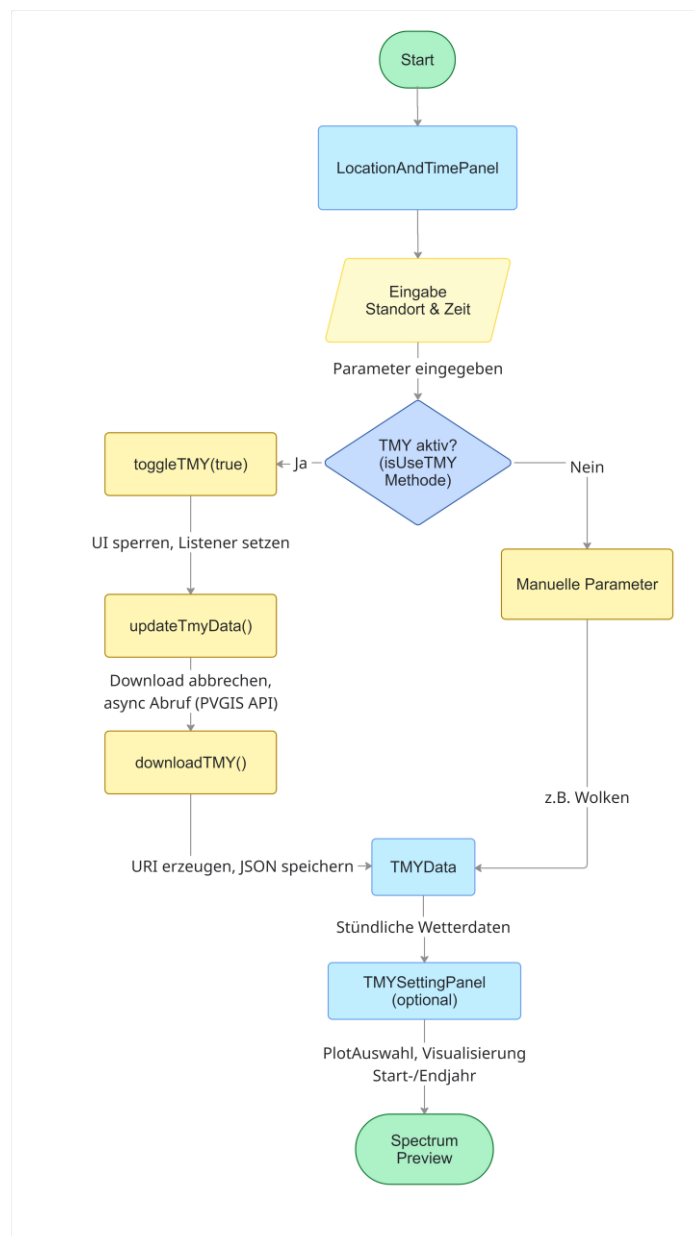


Abb. 12: GUI Workflow (eigene Abbildung)

Abb. 4 zeigt diesen Workflow in seiner Gesamtheit und verdeutlicht die Interaktion zwischen Benutzeroberfläche, Datenmodell und Simulationskernel. Die nachfolgenden Abschnitte beschreiben die beteiligten Klassen und Funktionen und ordnen sie jeweils in den dargestellten Workflow ein.

4.1 Zentrale Standort-, Zeit- und TMY-Verwaltung in der *LocationAndTimePanel* Klasse

Das *LocationAndTimePanel* dient als zentrales Interface zur Erfassung und Weiterverarbeitung geografischer und zeitabhängiger Parameter für Absorptions- und Spektrums-Berechnungen innerhalb der SETFOS-Simulationsumgebung. Damit stellt sie eine wesentliche Komponente für simulationsgestützte Analysen unter realitätsnahen Umweltbedingungen dar.

4.1.1 Aufgaben und Verantwortlichkeiten

Das Panel ermöglicht die strukturierte Eingabe und Verwaltung sämtlicher standortbezogenen Parameter, darunter geographische Breite und Länge sowie Höhenlage. Zusätzlich werden moduleigene Einstellungen wie Neigungswinkel und Ausrichtung erfasst. Ein weiterer zentraler Aufgabenbereich ist die Bereitstellung, Verwaltung und Nutzung von TMY-Datensätzen. Dies umfasst sowohl das Herunterladen und Einlesen der Daten als auch die Darstellung der zugehörigen Einstellungen in der Benutzeroberfläche.

Vor der Implementierung dieser Erweiterung besass dieses Panel bereits einen Abschnitt für Wetterparameter. Dieser umfasste jedoch nur einen einzigen Parameter, einen statischen Bewölkungsgrad. Dieser Parameter wurde als Alternative zu TMY-Daten beibehalten.

4.1.2 Architektur und technische Umsetzung

Die grafischen Strukturen basieren auf Java Swing und nutzen ein modulares Layout-Konzept mittels *SetfosMigLayout*. Die Datenbindung erfolgt über modellorientierte Klassen wie *SpectrumIllum* und *LocationAndTime*, welche die konsistente Synchronisation zwischen UI und interner Logik gewährleisten. Asynchrone Vorgänge, beispielsweise das Herunterladen von TMY-Datensätzen oder die Ausführung von Kernelprozessen, werden über *ScheduledExecutorService*, *CompletableFuture* und den internen *KernelWorker* gesteuert.

Für den Datenaustausch werden *Jakarta XML Binding* (JAXB) Objekte zur Generierung und Verarbeitung von XML-Dateien sowie *Gson* für das Einlesen der TMY-JSON-Strukturen eingesetzt. Reaktive Aktualisierungen erfolgen über das globale Ereignismanagement mittels *InputGlobals*, *EventBus* und *PropertyChangeListener*.

4.1.3 Ablauf und wichtige Funktionen

Der in Abb. 3 dargestellte Workflow wird im *LocationAndTimePanel* durch eine Abfolge klar definierter Verarbeitungsschritte umgesetzt. Diese Schritte bilden den vollständigen Ablauf von der Initialisierung der Benutzeroberfläche über die optionale Integration meteorologischer TMY-Daten bis hin zur Übergabe der vorbereiteten Eingaben an den Simulationskernel. Die beteiligten Methoden sind jeweils funktional den einzelnen Workflow-Phasen zugeordnet und werden im Folgenden zusammenhängend beschrieben.

1. Initialisierung der Benutzeroberfläche

Der Workflow beginnt mit der Initialisierung des Panels durch die Methode *initLocationTimePanel()*. In diesem Schritt werden sämtliche grafischen Komponenten erzeugt, interne Zustände initialisiert und die erforderlichen Listener registriert. Gleichzeitig wird der aktuelle Modellzustand ausgewertet, um den initialen UI-Zustand korrekt abzubilden.

Hierzu zählt insbesondere die Prüfung, ob dem aktuellen *LocationAndTime*-Objekt bereits eine TMY-Datei zugeordnet ist. Diese Prüfung erfolgt über die Methode *isUseTMY()* und bestimmt, ob der Workflow initial im TMY- oder im manuellen Eingabemodus startet.

2. Eingabe und Validierung der Standort- und Zeitparameter

Nach der Initialisierung erfolgt die Eingabe der standort- und zeitabhängigen Parameter durch den Benutzer. Änderungen an geographischer Position, Höhenlage sowie modulspezifischen Einstellungen werden kontinuierlich mit dem internen Datenmodell synchronisiert.

Die Validierung der Eingaben stellt sicher, dass der Workflow nur mit konsistenten Parametern fortgesetzt wird und bildet die Grundlage für nachgelagerte Berechnungs- und Abrufprozesse.

3. Verzweigung des Workflows abhängig von der TMY-Nutzung

Eine zentrale Entscheidungsstelle im Workflow ist die optionale Nutzung meteorologischer TMY-Daten. Diese Verzweigung wird durch die Methode *toggleTmy(...)* gesteuert.

Bei Aktivierung der TMY-Funktionalität werden TMY-spezifische Bedienelemente in der Benutzeroberfläche freigeschaltet, während manuelle Eingaben – beispielsweise zur Wolkenbedeckung – deaktiviert werden. Zusätzlich werden Listener registriert, welche Änderungen an den Standortparametern überwachen, da diese unmittelbar Einfluss auf die Gültigkeit der meteorologischen Daten haben. Anschließend wird der Abruf der TMY-Daten initiiert.

Bei Deaktivierung der TMY-Funktionalität werden laufende Abrufprozesse abgebrochen, alle TMY-bezogenen Listener entfernt und die zugehörige TMY-Datei aus dem Modell sowie dem Sitzungsverzeichnis gelöscht. Dadurch wird der Workflow wieder vollständig auf manuelle Eingaben zurückgeführt.

4. Asynchroner Abruf und Verwaltung der TMY-Daten

Der Abruf der meteorologischen Daten erfolgt asynchron und entspricht einer zentralen Phase des Workflows bei aktivierter TMY-Nutzung. Die Methode *updateTmyData(boolean delay)* koordiniert diesen Prozess. Vor dem Start eines neuen Abrufs werden laufende Downloadprozesse abgebrochen, um inkonsistente Zustände zu vermeiden. Abhängig vom übergebenen Parameter erfolgt der Abruf entweder verzögert oder unmittelbar.

Der eigentliche Datenabruf wird durch die Methode *downloadTMY()* durchgeführt. Diese erzeugt auf Basis der aktuellen Standortparameter eine Anfrage-URI, kommuniziert mit der PVGIS-API und speichert die empfangenen meteorologischen Daten als JSON-Datei im Sitzungsverzeichnis. Nach erfolgreichem Abschluss wird die Datei im *LocationAndTime*-Objekt registriert und steht dem weiteren Workflow zur Verfügung.

Die Erzeugung der Anfrage-URI ist dabei in die Methoden *getUri(...)* und *constructHttpUri(...)* ausgelagert, welche die Parameterzusammenstellung kapseln und als Abstraktionsschicht für zukünftige Erweiterungen dienen.

5. Optionale Analyse und Konfiguration der TMY-Daten

Nach erfolgreichem Abruf der TMY-Daten kann ein optionaler Analyse- und Konfigurationszweig des Workflows aktiviert werden. Dieser wird durch die Methode *clickedSettingTMY(JFrame)* eingeleitet.

In diesem Schritt wird die gespeicherte TMY-JSON-Datei geladen, in eine Liste von *TMYData*-Objekten deserialisiert und an das *TMYSettingPanel* übergeben. Dort können meteorologische Parameter grafisch analysiert, unterschiedliche Darstellungsformen gewählt und zeitliche Einschränkungen vorgenommen werden.

Dieser Schritt ist vollständig optional und beeinflusst den weiteren Workflow ausschließlich über die im Modell hinterlegten Einstellungen.

6. Bereinigung und Rückführung des Workflow-Zustands

Zur Aufrechterhaltung eines konsistenten Workflow-Zustands stellt die Methode *deleteTmy()* eine Bereinigungsfunktion bereit. Sie entfernt gespeicherte TMY-Dateien aus dem Sitzungsverzeichnis und setzt die entsprechenden Modellattribute zurück. Dieser Schritt wird insbesondere beim Deaktivieren der TMY-Funktionalität verwendet, um einen definierten Ausgangszustand für manuelle Eingaben wiederherzustellen.

4.2 Datenstruktur TMY in der TMYData Klasse darstellen

Die Record-Klasse *TMYData* dient als unveränderliche Datenstruktur zur Abbildung einzelner stündlicher Einträge eines TMY. Sie stellt meteorologische Parameter wie Temperatur, Luftfeuchte, Strahlungswerte und Windgrößen bereit und ist für die direkte JSON-Deserialisierung über *Gson* ausgelegt. Darüber hinaus

umfasst die Hilfsfunktionen zur Vereinheitlichung und Auswertung von Zeitstempeln, die für grafische Analysen oder zeitbasierte Vergleiche benötigt werden.

4.2.1 Aufgaben und Verantwortlichkeiten

TMYData repräsentiert einen einzelnen Datensatz einer meteorologischen Stunde und sorgt dafür, dass die relevanten Messgrößen in einer konsistenten und eindeutigen Form vorliegen. Durch die Verwendung von `@SerializedName` wird ein direktes Mapping der JSON-Schlüssel der PVGIS-API auf die Java-Felder ermöglicht. Ergänzend stellt die Klasse Methoden bereit, die den Zeitstempel standardisieren und dessen Komponenten extrahieren, was insbesondere für Visualisierungs- und Analysezwecke erforderlich ist.

4.2.2 Architektur und technische Umsetzung

Die Klasse liegt als Java-Record vor und ist damit unveränderlich. Wichtige Felder sind:

- **timeUTC** (String): Rohzeitstempel in UTC im von PVGIS bereitgestellten Format.
- **Meteorologische Messwerte** (double):
Temperatur (t2m), relative Luftfeuchte (rh), globale horizontale Strahlung (gh), direkte Strahlung (gdn), diffuse Strahlung (gdh), Infrarotstrahlung (irh), Windgeschwindigkeit (ws10m), Windrichtung (wd10m) sowie Luftdruck (sp).

Die Zuordnung erfolgt durch Gson-Annotationen wie `@SerializedName("G(h)")`.

4.2.3 Ablauf und wichtige Funktionen

Vor der grafischen Darstellung mit dem *TimeSettingPanel* werden sämtliche TMY-Datensätze in eine einheitliche Zeitrepräsentation überführt (siehe Abb. 3). Hierzu werden die Zeitstempel der einzelnen **TMYData**-Objekte standardisiert und in Objekte des Typs *Second* überführt.

4.3 Konfiguration und Analyse der TMYs in der TMYSettingsPanel Klasse

Das *TMYSetingPanel* dient als Benutzerschnittstelle zur Konfiguration und Analyse meteorologischer Datensätze eines TMYs. Die Komponente ermöglicht die Auswahl verschiedener Darstellungsformen, unterstützt die Anpassung der verwendeten Stichprobenjahre und stellt grafische Auswertungen zentraler meteorologischer Größen bereit. Grundlage der Visualisierung ist eine Liste von *TMYData*-Einträgen, welche stündlich aufgelöste Messwerte wie Bestrahlungsstärke, Temperatur oder Windgeschwindigkeit enthalten.

4.3.1 Aufgaben und Verantwortlichkeiten

Das *TMYSetingPanel* dient als grafische Komponente zur Darstellung, Konfiguration und Analyse von TMY-Daten innerhalb der Simulationsumgebung. Es ermöglicht die Auswahl unterschiedlicher meteorologischer Parameter, die Festlegung von Start- und Endjahren für die TMY-Auswertung sowie die visuelle Aufbereitung meteorologischer Zeitreihen. Darüber hinaus bindet das Panel die ausgewählten Einstellungen an die entsprechenden Modellparameter, sodass Änderungen unmittelbar in die Absorptions- und Spektrumsmodelle einfließen.

4.3.2 Architektur und technische Umsetzung

Die Komponente basiert auf Java Swing und wird über *MigLayout* strukturiert. Sie folgt einer klaren Trennung zwischen Steuerelementen und der modularen Erzeugung der Plots. Zur Darstellung der Zeitserien kommt *JFreeChart* zum Einsatz, ergänzt durch domänenspezifische Klassen wie *SetfosXYSeries* zur Datenmodellierung. Die Zeitinformation der TMY-Daten wird durch *TMYData* standardisiert, siehe dazu Kapitel 4.2.3.

Interaktive Elemente, wie Auswahlboxen oder Textfelder, sind über Listener angebunden und lösen intern eine Neuberechnung oder Neuvisualisierung der Daten aus. Auf Fehlersituationen, etwa unparsebare Zeitstempel, reagiert die Komponente robust, indem problematische Datenpunkte ignoriert werden, ohne die Gesamtgenerierung zu unterbrechen.

4.3.3 Ablauf und wichtige Funktionen

Das *TimeSettingPanel* bildet innerhalb des in Abb. 3 dargestellten Gesamtworkflows die optionale Analyse- und Visualisierungsphase der meteorologischen TMY-Daten. Es wird ausschließlich dann aktiviert, wenn im *LocationAndTimePanel* die Nutzung von TMY-Daten gewählt wurde und ein gültiger Datensatz erfolgreich abgerufen wurde.

Nach der Initialisierung übernimmt das Panel die vollständige Konfiguration und Darstellung der verfügbaren TMY-Visualisierungen. Der Ablauf ist dabei vollständig reaktiv ausgelegt und reagiert unmittelbar auf Benutzerinteraktionen. Die zentrale Ablaufstruktur lässt sich wie folgt zusammenfassen:

- 1. Erzeugung der Benutzeroberfläche**

Zu Beginn werden sämtliche Eingabe- und Auswahlkomponenten aufgebaut. Dazu zählen insbesondere Auswahlfelder für den darzustellenden meteorologischen Parameter sowie Eingabefelder für Start- und Endjahr der TMY-Messperiode. Diese Parameter sind direkt an das zugrunde liegende Absorptions- bzw. Spektrumsmodell gebunden und beeinflussen die nachgelagerte Auswertung im Workflow.

- 2. Auswahl eines Plottyps**

Der Benutzer wählt den gewünschten Darstellungsparameter, beispielsweise Sonnenstrahlung, Temperatur, relative Luftfeuchte oder Windgeschwindigkeit. Jede Änderung des Plottyps stellt einen neuen Analysepfad innerhalb des Workflows dar und führt unmittelbar zur Neuberechnung der darzustellenden Zeitreihe.

- 3. Erstellung und Aktualisierung der Vorschauplots**

Bei jeder relevanten Änderung wird der aktuell dargestellte Plot vollständig entfernt und durch einen neu erzeugten Chart ersetzt. Dadurch wird sichergestellt, dass die Visualisierung stets konsistent mit dem aktuellen Modell- und Parameterzustand ist und keine inkrementellen Artefakte entstehen.

Durch diesen Ablauf ermöglicht das *TimeSettingPanel* eine robuste, interaktive und benutzerfreundliche Analyse realer meteorologischer Datensätze.

5 Kernel-Code-Implementation

Nachdem die Implementierung der GUI-Tasks abgeschlossen war, erfolgte die Erweiterung des Kernels in C++. Ziel dieser Arbeiten war es, das bestehende Simulationssystem um die Fähigkeit zu erweitern, **meteorologische Zeitreihendaten aus Typical Meteorological Year (TMY) Dateien** einzulesen, zu verarbeiten und konsistent in die Energieertragsberechnung zu integrieren.

Die Kernel-Erweiterungen umfassen insbesondere:

- das Auslesen TMY-spezifischer Parameter aus .par-Dateien,
- das Parsen und Validieren von TMY-Dateien,
- die Speicherung und zeitliche Interpolation meteorologischer Daten,
- sowie die Einbindung dieser Daten in die bestehende Ertragsberechnungslogik.

Die Implementierung folgt dabei einem klar strukturierten, mehrstufigen Workflow, der vom Einlesen der Konfigurationsparameter über das Parsen und die interne Datenhaltung bis hin zur Nutzung der TMY-Daten in der Simulation reicht. Dieser Ablauf ist in Abb. 13 dargestellt, welches die beteiligten Komponenten und deren Zusammenspiel verdeutlicht. Das Schema dient als konzeptionelle Orientierung für die nachfolgenden Unterkapitel und erleichtert das Verständnis der einzelnen Implementierungsschritte sowie ihrer Abhängigkeiten.

Die folgenden Abschnitte beschreiben die vorgenommenen Änderungen und Neuentwicklungen im Kernel detailliert und orientieren sich dabei an der im

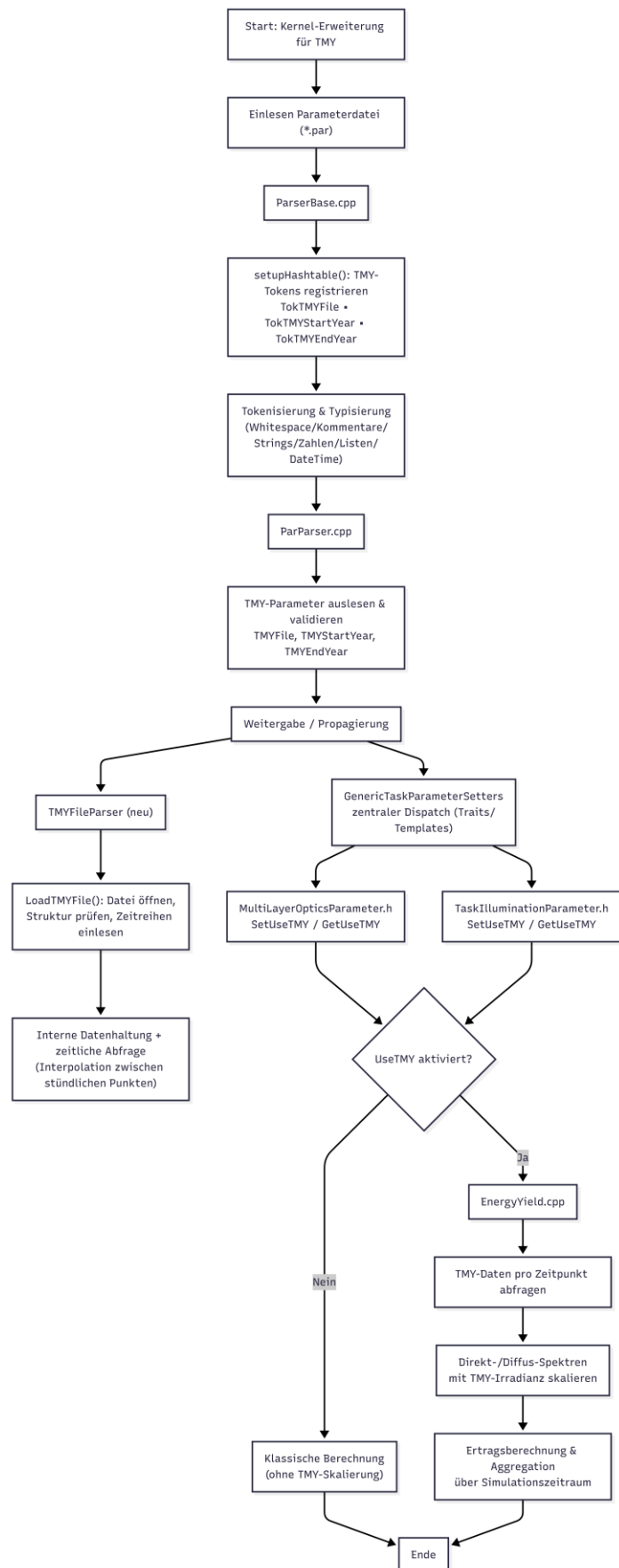


Abb. 13: Kernel Workflow (eigene Abbildung)

Schema dargestellten Abfolge der Verarbeitungsschritte.

5.1 Erweiterung der ParserBase.cpp zur Unterstützung von TMY-Parametern

Im ersten Schritt der Kernel-Erweiterung wurde die bestehende Parser-Infrastruktur angepasst, um TMY-spezifische Parameter aus den Konfigurationsdateien verarbeiten zu können. Da die *ParParser*-Klasse (siehe Kapitel 5.2) auf der *ParserBase*-Klasse aufbaut, stellt diese Erweiterung die grundlegende Voraussetzung für die nachgelagerte Verarbeitung von TMY-Parameter dar.

Die folgenden Abschnitte beschreiben die Aufgaben, die architektonische Umsetzung sowie den konkreten Ablauf der *ParserBase* im Rahmen der TMY-Integration detailliert.

5.1.1 Aufgaben und Verantwortlichkeiten

Die Klasse *ParserBase* stellt die grundlegende Infrastruktur für das Parsen von Konfigurationsdateien bereit und bildet die Basis aller spezialisierten Parser im System. Ihre Hauptaufgabe besteht darin, Tokenstrings zu verwalten, Eingabedaten zu tokenisieren und typisierte Werte aus Parameterdateien bereitzustellen.

Im Kontext der TMY-Integration übernimmt die *ParserBase* eine zentrale Rolle, da sie die TMY-spezifischen Parameter (*TMYFile*, *TMYStartYear*, *TMYEndYear*) systemweit verfügbar macht. Wie im Workflow-Schema dargestellt, erfolgt dieser Schritt unmittelbar nach dem Einlesen der Parameterdatei (*.par) und bildet damit die Grundlage für alle nachfolgenden Verarbeitungsschritte.

Die *ParserBase* selbst führt keine inhaltliche Interpretation der TMY-Daten durch, sondern stellt sicher, dass die entsprechenden Tokens konsistent erkannt und an nachgelagerte Parserkomponenten weitergereicht werden.

5.1.2 Architektur und technische Umsetzung

Architektonisch fungiert *ParserBase* als zentrale Tokenisierungs- und Parsing-Komponente, auf der spezialisierte Parser wie der *ParParser* aufbauen. Die Token-Zuordnung erfolgt über eine statisch initialisierte bidirektionale Hashtable, welche String-Repräsentationen eindeutig einem internen *TokenEnum* zuordnet und umgekehrt.

Die Initialisierung dieser Token-Registry erfolgt einmalig über die Methode `setupHashtable()`. Im Rahmen der TMY-Erweiterung wurde diese Registry um zusätzliche Token ergänzt, die im Workflow-Schema dem Übergang von der Parameterdatei zur *ParserBase* entsprechen.

Der Lesealgorithmus verarbeitet:

- Whitespace und Kommentare,
- Strings in Anführungszeichen,
- numerische Werte,
- Listentokens,
- sowie komplexe Datentypen wie *DateTime* oder *Dateiinformationen*.

Nicht erkannte Eingaben werden standardmäßig als String-Tokens behandelt. Datei- und Pfadangaben werden plattformabhängig normalisiert, während die Verarbeitung physikalischer Einheiten über die bestehende *UnitTable* erfolgt.

Die Fehlerbehandlung erfolgt konsistent über *ReturnStatus* anstelle von *Exceptions*. Zusätzlich werden Positionsinformationen (Datei, Zeile, Spalte) erfasst, um präzise Fehlermeldungen zu ermöglichen. Diese robuste Architektur stellt sicher, dass fehlerhafte TMY-Parameter frühzeitig erkannt werden, bevor sie – wie im Schema gezeigt – an den *ParParser* oder weitere Komponenten weitergegeben werden.

5.1.3 Ablauf und wichtige Funktionen

Der Ablauf innerhalb der *ParserBase* ist eng an den im Schema dargestellten Initialisierungspfad gekoppelt:

1. **Einlesen der Parameterdatei (*.par)**
Die *ParserBase* wird als erste Parserinstanz aktiv, nachdem die Parameterdatei geladen wurde.
2. **Initialisierung der Tokenstrings**
Während der Initialisierung ruft das System `setupHashtable()` auf. Hier werden neben den bestehenden Tokens auch die neuen TMY-spezifischen Tokens registriert:
 - `TokTMYFile`
 - `TokTMYStartYear`
 - `TokTMYEndYear`
3. **Tokenisierung der Eingabedaten**
Die Parameterdatei wird sequenziell gelesen und in Tokens zerlegt. TMY-relevante Parameter werden dabei korrekt erkannt und typisiert.
4. **Weitergabe an nachgelagerte Parser**
Die erzeugten Tokens stehen anschliessend allen abgeleiteten Parsern, insbesondere dem *ParParser*, zur Verfügung. Im Workflow-Schema ist dieser Übergang explizit zwischen *ParserBase* und *ParParser* dargestellt.

Durch diese Erweiterung stellt die *ParserBase* sicher, dass TMY-Parameter **nahtlos in den bestehenden Parsing-Workflow integriert** sind, ohne die Architektur oder das Verhalten bestehender Parser zu verändern.

5.2 Erweiterung des *ParParser.cpp* zur Verarbeitung von TMY-Konfigurationsparametern

Aufbauend auf der in Abschnitt 5.1 beschriebenen Erweiterung der *ParserBase* übernimmt der **ParParser** die eigentliche inhaltliche Auswertung der aus der Parameterdatei (*.par) gelesenen Tokens. Er bildet die zentrale Schnittstelle zwischen der syntaktischen Tokenisierung und der Initialisierung der internen Simulationsdatenstrukturen.

Wie im Workflow-Schema dargestellt, folgt der *ParParser* unmittelbar auf die *ParserBase* und ist dafür verantwortlich, TMY-spezifische Parameter auszulesen, zu validieren und an die entsprechenden Systemkomponenten weiterzuleiten. Durch seine Erweiterung wird es möglich, meteorologische Zeitreihendaten direkt in der Konfigurationsdatei zu referenzieren und kontrolliert in den Simulationsablauf einzubinden.

5.2.1 Aufgaben und Verantwortlichkeiten

Der *ParParser* ist für das Einlesen konkreter Parameter aus den Konfigurationsdateien zuständig und überführt diese in die internen Datenstrukturen der Simulation. Seine Hauptverantwortung liegt darin, die in der Parameterdatei definierten Einstellungen semantisch korrekt zu interpretieren und konsistent zu verarbeiten.

Im Zuge der TMY-Integration wurde der *ParParser* erweitert, um TMY-bezogene Einstellungen wie Dateipfade sowie Start- und Endjahre auszuwerten. Entsprechend dem Workflow-Schema übernimmt er die Aufgabe, diese Informationen aus dem Tokenstrom zu extrahieren und sowohl an Task- und Optikparameter als auch an den spezialisierten *TMYFileParser* weiterzugeben.

Damit fungiert der *ParParser* als zentrale Steuerinstanz, welche die Nutzung von TMY-Daten konfigurationsseitig aktiviert und deren korrekte Einbindung in die Simulation vorbereitet.

5.2.2 Architektur und technische Umsetzung

Architektonisch ist der *ParParser* der zentrale Abschnitts-Parser für .par-Dateien und baut vollständig auf der tokenbasierten Infrastruktur der *ParserBase* auf. Er verarbeitet die Hauptsektionen der Parameterdatei (Version, Structure, Define, Commands und Options) in einer festen, deterministischen Reihenfolge.

Die eingelesenen Parameter werden in die internen Simulationsdatenstrukturen überführt, darunter unter anderem:

- *Simulation*
- *SimulationParameter*
- *MultiLayerStructure*
- *MultiLayerOpticsParameter*

Der Parser arbeitet sequenziell mit *ReadToken()* und verwendet *ReturnStatus* für eine einheitliche Fehlerbehandlung, einschliesslich präziser Positionsangaben (Datei, Zeile, Spalte).

Für komplexere Konfigurationsbereiche bindet der *ParParser* spezialisierte Sub-Parser ein, beispielsweise für Materialmodelle, Streuung, Refraktivitätsdaten und Zeitreihendaten. Dateipfade werden relativ zur aktuellen Parameterdatei aufgelöst und über den *InputFileManager* eingebunden. Nach strukturellen Änderungen stellt der Parser sicher, dass zugehörige Sweep- und Optimierungsparameter konsistent aktualisiert werden.

Diese Architektur ermöglicht es, die neuen TMY-Parameter nahtlos in den bestehenden Parsing-Prozess zu integrieren, wie es im Workflow-Schema durch die Verzweigung vom *ParParser* zu den nachgelagerten Komponenten dargestellt ist.

5.2.3 Ablauf und wichtige Funktionen

Der Ablauf des *ParParser* orientiert sich eng am im Schema dargestellten Verarbeitungsfluss:

1. Übernahme der Tokens aus der *ParserBase*

Nach Abschluss der Tokenisierung stehen dem *ParParser* sämtliche Parameter, einschliesslich der TMY-spezifischen Tokens, zur Verfügung.

2. Einlesen und Validierung der TMY-Parameter

Die TMY-bezogenen Angaben (*TMYFile*, *TMYStartYear*, *TMYEndYear*) werden erkannt, geprüft und intern gespeichert.

3. Weitergabe der Parameter

Die ausgelesenen Informationen werden:
an Task- und Optikparameter weitergereicht, über generische Setter propagiert, und an den *TMYFileParser* zur weiteren Verarbeitung übergeben.

4. Vorbereitung der Simulation

Durch das Setzen entsprechender Flags wird festgelegt, ob die Simulation, wie im Entscheidungsknoten des Schemas gezeigt, eine klassische oder eine TMY-basierte Berechnung durchführt.

Durch diese Erweiterungen stellt der *ParParser* sicher, dass TMY-Daten konfigurationsseitig konsistent aktiviert, validiert und systemweit verfügbar gemacht werden.

5.3 Erweiterung von *MultiLayerOpticsParameter.h* zur Steuerung der TMY-Nutzung

Nach dem Einlesen und der semantischen Verarbeitung der TMY-Parameter durch den *ParParser* werden die entsprechenden Konfigurationsinformationen in den optischen Parameterstrukturen abgelegt. Die Header-Datei *MultiLayerOpticsParameter.h* definiert dabei die Schnittstelle zur Verwaltung dieser Parameter und stellt sie der nachgelagerten optischen Simulationslogik zur Verfügung.

Wie im Workflow-Schema dargestellt, erfolgt in diesem Schritt die Übergabe der TMY-Konfigurationsinformation vom Parsing-Bereich in den optischen Simulationskontext. Die Erweiterung des Header-Files schafft die Voraussetzung dafür, dass die Nutzung von TMY-Daten eindeutig konfigurierbar und systemweit konsistent abrufbar ist.

5.3.1 Aufgaben und Verantwortlichkeiten

Die Klasse `MultiLayerOpticsParameter`, deren Schnittstelle in der Header-Datei `MultiLayerOpticsParameter.h` definiert ist, ist für die Verwaltung sämtlicher optischer Parameter eines Mehrschichtsystems verantwortlich. Sie dient als reine Parameter- und Zustandsklasse und enthält keine numerische Simulationslogik.

Im Zuge der TMY-Integration wurde die Schnittstelle um ein zusätzliches TMY-Steuerungsflag erweitert. Dieses Flag legt fest, ob meteorologische Zeitreihendaten im weiteren Simulationsverlauf berücksichtigt werden sollen. Die Klasse übernimmt damit folgende Verantwortlichkeiten:

- zentrale Speicherung optischer Konfigurationsparameter,
- Bereitstellung einer klar definierten Schnittstelle zur Aktivierung bzw. Deaktivierung der TMY-Nutzung,
- Weitergabe der Konfigurationsinformation an die optische Simulationslogik.

Im Workflow-Schema ist diese Rolle als Übergang vom `ParParser` zu den optischen Parametern dargestellt und bildet die Grundlage für die spätere Entscheidungslogik.

5.3.2 Architektur und technische Umsetzung

Die Erweiterung der Header-Datei erfolgt über die Definition zweier zusätzlicher öffentlicher Methoden:

- `SetUseTMY(bool)`
- `GetUseTMY()`

Diese Methoden definieren die **Schnittstelle** zur Konfiguration der TMY-Nutzung. Die eigentliche Implementierung der Methoden erfolgt in der zugehörigen Implementierungsdatei, während der Header ausschliesslich die Signatur und die semantische Bedeutung des Flags festlegt.

Die Klasse selbst trifft keine Entscheidung über das Strahlungs- oder Berechnungsmodell und enthält keine Logik zur Auswertung der TMY-Daten. Stattdessen wird das gesetzte Flag von der übergeordneten Simulationslogik abgefragt, wie im Workflow-Schema nach der Simulationsinitialisierung dargestellt.

Durch diese Gestaltung bleibt die Klasse:

- schlank und zustandsorientiert,
- klar von der Implementierungslogik getrennt,
- gut testbar und leicht erweiterbar.

5.3.3 Ablauf und wichtige Funktionen

Im Gesamtablauf der Simulation nimmt die in `MultiLayerOpticsParameter.h` definierte Schnittstelle folgende Rolle ein:

1. Initialisierung der Parameter

Während des Parsings werden die TMY-bezogenen Einstellungen aus der Parameterdatei übernommen und über `SetUseTMY(bool)` im Parameterobjekt gespeichert.

2. Persistente Bereitstellung der Konfiguration

Das gesetzte Flag ist über die gesamte Simulationsinitialisierung hinweg verfügbar und kann jederzeit über `GetUseTMY()` abgefragt werden.

3. Nutzung durch die Simulationslogik

Die Entscheidung über die Verwendung eines klassischen oder eines TMY-basierten Strahlungsmodells erfolgt in der nachgelagerten optischen Simulationslogik, welche das Flag aus den optischen Parametern ausliest.

Damit stellt die Header-Datei `MultiLayerOpticsParameter.h` eine klar definierte, konfigurationsseitige Schnittstelle bereit, die, wie im Workflow-Schema dargestellt, den Übergang von der Parameterauswertung zur eigentlichen Simulationsentscheidung ermöglicht.

5.4 Erweiterung der `TaskIlluminationParameter.h` zur Steuerung der TMY-Nutzung

Neben den optischen Parametern müssen auch die Parameter der Beleuchtungsaufgaben in der Lage sein, die Nutzung von TMY-Daten zu steuern. Die Header-Datei `TaskIlluminationParameter.h` definiert die Schnittstelle für diese Aufgabenparameter und stellt sicher, dass die entsprechende Konfigurationsinformation systemweit verfügbar ist.

Wie im Workflow-Schema dargestellt, erfolgt an dieser Stelle die Weitergabe der TMY-Konfiguration an die Task-Ebene der Simulation. Durch die Erweiterung der Header-Datei wird gewährleistet, dass Beleuchtungsaufgaben konsistent auf die gleiche TMY-Entscheidung zugreifen wie die optische Simulation.

5.4.1 Aufgaben und Verantwortlichkeiten

Die Klasse `TaskIlluminationParameter` verwaltet die Parameter für Beleuchtungsaufgaben innerhalb der Simulation. Sie dient als reine Parameterklasse und enthält keine Logik zur eigentlichen Ausführung der Aufgaben.

Analog zu den optischen Parametern wurde die Schnittstelle um ein zusätzliches TMY-Steuerungsflag erweitert. Dieses Flag legt fest, ob für Beleuchtungsaufgaben meteorologische Zeitreihendaten aus TMY-Dateien berücksichtigt werden sollen.

Die Hauptverantwortlichkeiten der Klasse umfassen:

- die zentrale Speicherung von Task-spezifischen Beleuchtungsparametern,
- die Bereitstellung einer einheitlichen Schnittstelle zur Konfiguration der TMY-Nutzung,
- sowie die Weitergabe der Konfigurationsinformation an die ausführende Task-Logik.

Im Workflow-Schema ist diese Erweiterung als paralleler Pfad zur optischen Parameterauswertung dargestellt.

5.4.2 Architektur und technische Umsetzung

Die Erweiterung der Header-Datei erfolgt durch die Definition zweier zusätzlicher öffentlicher Methoden:

- `SetUseTMY(bool)`
- `GetUseTMY()`

Diese Methoden definieren die Schnittstelle zur Aktivierung oder Deaktivierung der TMY-Nutzung für Beleuchtungsaufgaben. Die Klasse selbst trifft keine Entscheidung über die konkrete Verwendung der TMY-Daten, sondern stellt das Flag der übergeordneten Task-Logik zur Verfügung.

Durch diese Architektur bleibt die Klasse:

- klar auf die Parameterhaltung beschränkt,
- unabhängig von der Implementierung der Aufgabenlogik,
- und konsistent zu den übrigen TMY-relevanten Parameterklassen.

5.4.3 Ablauf und wichtige Funktionen

Im Gesamtablauf der Simulation übernimmt die in `TaskIlluminationParameter.h` definierte Schnittstelle folgende Rolle:

1. Initialisierung der Task-Parameter

Während des Parsings werden die TMY-bezogenen Einstellungen aus der Konfigurationsdatei übernommen und über `SetUseTMY(bool)` im Parameterobjekt gespeichert.

2. Weitergabe innerhalb der Task-Ebene

Das gesetzte Flag ist während der gesamten Initialisierungs- und Ausführungsphase verfügbar und kann über `GetUseTMY()` abgefragt werden.

3. Konsistente Nutzung in der Simulation

Durch die einheitliche Schnittstelle wird sichergestellt, dass Beleuchtungsaufgaben dieselbe TMY-Konfigurationsentscheidung verwenden wie die optische Simulation, wie es im Workflow-Schema vorgesehen ist.

5.5 Erweiterung von `GenericTaskParameterSetters.cpp` und `.h` zur einheitlichen Initialisierung der TMY-Nutzung

Zur Sicherstellung einer konsistenten Initialisierung der TMY-Nutzung über unterschiedliche Task-Typen hinweg wurden die `GenericTaskParameterSetters` erweitert. Diese Komponente stellt generische Setter-Funktionen bereit, mit denen Task-Parameter unabhängig vom konkreten Task-Typ initialisiert werden.

Im Workflow-Schema übernehmen die `GenericTaskParameterSetters` die Rolle einer zentralen Verteilstelle, über welche die im `ParParser` ermittelten TMY-Einstellungen einheitlich an alle relevanten Task-Parameter weitergegeben werden.

5.5.1 Aufgaben und Verantwortlichkeiten

Die Klasse `GenericTaskParameterSetters` stellt generische Initialisierungs- und Setter-Funktionen für unterschiedliche Task-Parameter bereit. Sie dienen dazu, Konfigurationsinformationen zentral zu setzen, ohne task-spezifische Logik duplizieren zu müssen.

Im Zuge der TMY-Integration wurde diese Klasse erweitert, um auch das TMY-Steuerungsflag für unterschiedliche Task-Parameter setzen zu können. Dadurch übernimmt sie folgende Verantwortlichkeiten:

- Zentrale Initialisierung der TMY-Nutzung für verschiedene Task-Typen,
- Vermeidung redundanter Initialisierungslogik,
- Sicherstellung einer einheitlichen Konfiguration über alle Simulationstypen hinweg.

5.5.2 Architektur und technische Umsetzung

Die `GenericTaskParameterSetters` sind Teil einer modularen, datenorientierten Architektur mit klar getrennten Zuständigkeiten zwischen Parsing, Domänenlogik und parametrisierter Verarbeitung. Sie fungieren als Adapter-Schicht zwischen den vom Parser erzeugten generischen Parameterrepräsentationen und den konkreten Task- und Domain-Objekten der Simulation.

Die Eingaben werden zunächst durch den Parser in strukturierte Objekte wie `SimulationParameter` und `ParameterIdentifier` überführt. Die `GenericTaskParameterSetters` arbeiten ausschliesslich auf diesen abstrahierten Repräsentationen und bleiben damit vollständig von der syntaktischen Eingabeverarbeitung entkoppelt. Die eigentliche Simulationslogik ist in Domain-Modellen gekapselt, welche keine Kenntnis über Parsing- oder Setter-Mechanismen besitzen.

Technisch basiert die Setter-Schicht auf einem Trait- und Template-basierten Design. Für jeden Parametertyp existiert eine Spezialisierung von `GenericTaskParameterTraits`, welche den Werttyp sowie die Logik zur Erzeugung, Validierung und Anwendung des Parameters definiert. Der Dispatch zur passenden Implementierung erfolgt vollständig zur Compile-Time, wodurch Laufzeit-Overhead vermieden wird.

Im Kontext der TMY-Integration ermöglicht diese Architektur eine zentrale und einheitliche Initialisierung des TMY-Steuerungsflags für unterschiedliche Task-Typen. Das Flag wird einmal aus den Konfigurationsdaten abgeleitet und anschliessend über die `GenericTaskParameterSetters` konsistent an alle relevanten Task-Parameter propagiert. Dadurch wird sichergestellt, dass sämtliche Tasks innerhalb der Simulation dieselbe Entscheidung bezüglich der Nutzung von TMY-Daten treffen, was eine konsistente und nachvollziehbare TMY-basierte Berechnung ermöglicht.

5.5.3 Ablauf und wichtige Funktionen

Im Ablauf der Simulation übernehmen die *GenericTaskParameterSetters* folgende Aufgaben:

1. **Übernahme der TMY-Konfiguration**

Die im *ParParser* ausgelesenen TMY-Einstellungen werden an die generischen Setter übergeben.

2. **Einheitliches Setzen der Task-Parameter**

Die Setter propagieren das TMY-Flag an alle relevanten Task-Parameter, einschliesslich der Beleuchtungsaufgaben.

3. **Sicherstellung der Konsistenz**

Durch diese zentrale Initialisierung wird gewährleistet, dass alle Tasks – unabhängig von ihrem Typ – dieselbe Entscheidung bezüglich der Nutzung von TMY-Daten treffen.

Damit bilden die *GenericTaskParameterSetters* einen wichtigen Baustein im Workflow-Schema, um die konsistente und skalierbare Nutzung von TMY-Daten im gesamten Simulationssystem sicherzustellen.

5.6 Einführung des *TMYFileParser* zur Verarbeitung meteorologischer Zeitreihendaten

Nach der konfigurationsseitigen Aktivierung der TMY-Nutzung und der Propagierung der entsprechenden Flags an die relevanten Simulationsparameter erfolgt der eigentliche Zugriff auf die meteorologischen Zeitreihendaten. Diese Aufgabe übernimmt der neu eingeführte *TMYFileParser*.

Wie im Workflow-Schema dargestellt, ist der *TMYFileParser* die zentrale Komponente zur Entkopplung externer Wetterdaten von der Simulationslogik. Er stellt sicher, dass TMY-Dateien unabhängig vom restlichen Parsingsystem eingelesen, validiert und in einer strukturierten Form bereitgestellt werden. Damit bildet er die technische Grundlage für die anschliessende Nutzung der TMY-Daten in der Energieertragsberechnung.

5.6.1 Aufgaben und Verantwortlichkeiten

Der *TMYFileParser* ist ein neu implementiertes Modul zur Verarbeitung von TMY-Dateien. Seine Verantwortung ist klar abgegrenzt: Er übernimmt ausschliesslich das Einlesen, Validieren, Speichern und Bereitstellen meteorologischer Zeitreihendaten und enthält keinerlei simulations- oder ertragsbezogene Logik.

Im Gesamtworkflow fungiert der *TMYFileParser* als Brücke zwischen externen Wetterdatensätzen und der internen Simulationslogik. Er stellt sicher, dass TMY-Daten in einer strukturierten und konsistenten Form zur Verfügung stehen, bevor sie in der Energieertragsberechnung verwendet werden.

5.6.2 Architektur und technische Umsetzung

Architektonisch ist der *TMYFileParser* in die bestehende Parser-Schicht integriert und wird vom *ParParser* initialisiert, sobald TMY-Parameter in der Konfigurationsdatei erkannt werden. Er nutzt dieselben grundlegenden Infrastrukturkomponenten wie andere Parser-Module, etwa das zentrale Fehler- und Statusreporting über *ReturnStatus* sowie die Dateiverwaltung über den *InputFileManager*.

Die Implementierung folgt dem allgemeinen Parser-Design des Systems, verzichtet jedoch bewusst auf eine tiefe Kopplung an die übrige Konfigurationslogik. Der *TMYFileParser* arbeitet dateibasiert, prüft die strukturelle Integrität der TMY-Datei und überführt die eingelesenen Werte in interne, datenorientierte Strukturen. Diese sind auf effiziente zeitliche Abfragen ausgelegt und ermöglichen das einfache Interpolieren zwischen den stündlichen Datenpunkten.

Durch diese klare Trennung bleibt der Parser schlank, gut testbar und leicht erweiterbar, beispielsweise für weitere meteorologische Datenpunkte die in der momentanen Implementation bewusst nicht eingelesen werden.

5.6.3 Ablauf und wichtige Funktionen

Der Ablauf des TMYFileParser ist bewusst einfach gehalten:

1. **Laden der TMY-Datei**

Die Funktion LoadTMYFile() öffnet die referenzierte Datei, validiert deren Aufbau und liest die enthaltenen meteorologischen Zeitreihen ein.

2. **Interne Aufbereitung der Daten**

Die Rohdaten werden in interne Datenstrukturen überführt, welche eine effiziente zeitbasierte Abfrage ermöglichen.

Damit stellt der TMYFileParser, wie im Workflow-Schema dargestellt, die zentrale Zugriffsschicht auf meteorologische Daten dar.

5.7 Erweiterung von EnergyYield.cpp zur TMY-basierten Energieertragsberechnung

Nachdem die meteorologischen Zeitreihendaten durch den *TMYFileParser* eingelesen und bereitgestellt wurden, erfolgt deren konkrete Nutzung in der Energieertragsberechnung. Diese Aufgabe wird in der Klasse *EnergyYield* umgesetzt, welche den abschliessenden Verarbeitungsschritt im TMY-Workflow darstellt.

Wie im Workflow-Schema dargestellt, bildet *EnergyYield* den Endpunkt der TMY-Integration: Hier fließen die konfigurationsseitig aktivierten TMY-Daten erstmals direkt in numerische Berechnungen ein und ermöglichen eine realitätsnahe Ermittlung des Energieertrags unter zeitlich variierenden Umweltbedingungen.

5.7.1 Aufgaben und Verantwortlichkeiten

Die Klasse **EnergyYield** ist für die **Berechnung des Energieertrags der Solarzelle** zuständig. Sie verarbeitet spektrale Eingangsgrössen, optische Ergebnisse sowie Beleuchtungsparameter und führt diese zu einer Gesamtenergie- bzw. Ertragsgrösse zusammen.

Im Rahmen der Erweiterung wurde EnergyYield um die Fähigkeit ergänzt, **meteorologische Daten aus Typical Meteorological Year (TMY) Dateien** zu berücksichtigen. Dadurch kann der Energieertrag zeitabhängig skaliert und unter realistischen Einstrahlungsbedingungen berechnet werden. Die Klasse übernimmt dabei keine Aufgaben des Dateiparsings, sondern nutzt ausschliesslich die vom TMYFileParser bereitgestellten Daten.

5.7.2 Architektur und technische Umsetzung

Architektonisch ist EnergyYield Teil der **fachlichen Kernlogik** der Simulation und klar von Parsing, Parameterinitialisierung und Optik-Orchestrierung getrennt. Die Klasse greift auf vorbereitete Domain-Objekte wie optische Strukturen, Spektren und Task-Parameter zu und führt die eigentlichen numerischen Berechnungen durch.

Die TMY-Integration erfolgt über eine **lose Kopplung an den TMYFileParser**. Für einen gegebenen Zeitpunkt wird aus der TMY-Zeitreihe ein passender Index bestimmt, auf dessen Basis die spektralen Intensitäten skaliert werden. Die eigentliche Spektrenberechnung (Direkt- und Diffusanteile) bleibt dabei unverändert und wird lediglich um einen TMY-basierten Skalierungsfaktor ergänzt.

Das Design folgt weiterhin einem datenorientierten Ansatz mit klaren Zuständigkeiten:

- Parsing und Datenhaltung in Parser- und I/O-Modulen,
- Konfigurationspropagierung über Parameter- und Setter-Strukturen,
- numerische Berechnung und Aggregation ausschliesslich in EnergyYield.

Diese Trennung ermöglicht eine saubere Integration der TMY-Daten ohne strukturelle Änderungen an bestehenden Berechnungsmodellen.

5.7.3 Ablauf und wichtige Funktionen

Der Ablauf der TMY-basierten Energieertragsberechnung gestaltet sich wie folgt:

1. Initialisierung

Nach Abschluss der Simulationsinitialisierung wird geprüft, ob die TMY-Nutzung aktiviert ist.

2. Abfrage der TMY-Daten

Über die Schnittstelle zum TMYFileParser werden für die jeweiligen Zeitpunkte die relevanten meteorologischen Daten abgerufen.

3. Skalierung der Spektren

Die berechneten Direkt- und Diffusspektren werden entsprechend der TMY-Irradianzen skaliert und als aktualisiert markiert.

4. Ertragsberechnung

Die zeitabhängig skalierten Spektren fließen in die bestehende Ertragslogik ein und werden über den Simulationszeitraum aggregiert.

Damit schliesst die Klasse EnergyYield – wie im Workflow-Schema dargestellt – die TMY-Integration ab und ermöglicht eine **konsistente, zeitaufgelöste und realitätsnahe Energieertragsberechnung**.

6 Resultate

Die im Rahmen dieser Arbeit erzielten Resultate umfassen sowohl Erweiterungen der grafischen Benutzeroberfläche als auch substanzielle Anpassungen im Simulationskernel. Ziel war es, meteorologische Zeitreihendaten eines Typical Meteorological Year (TMY) durchgängig in den Simulationsworkflow zu integrieren und sowohl für Anwender als auch für die Berechnungslogik nutzbar zu machen.

Die folgenden Abschnitte fassen die erzielten Ergebnisse getrennt nach GUI- und Kernel-Ebene zusammen und zeigen, wie die neu eingeführten Funktionalitäten ineinandergreifen.

6.1 GUI

Die Erweiterung der grafischen Benutzeroberfläche führte zu einer vollständigen Integration der TMY-Funktionalität in das bestehende *LocationAndTimePanel* und ergänzende Komponenten. Durch die neu implementierten Steuer-, Anzeige- und Visualisierungselemente kann SETFOS nun meteorologische Datensätze eines Typical Meteorological Year effizient nutzen, verwalten und grafisch darstellen. Die wichtigsten Ergebnisse werden im Folgenden zusammengefasst.

6.1.1 Neuer Workflow zur Verwendung von TMY-Daten

Abb. 14: Erweiterung mit CheckBox Use TMY (eigene Abbildung)

Die Benutzeroberfläche wurde um eine Checkbox *Use TMY* erweitert (Siehe Abb.5), welche als zentrale Schaltstelle für die Aktivierung und Deaktivierung der TMY-Funktion dient. Wird die Option aktiviert,

erscheinen zusätzliche Einstellmöglichkeiten, insbesondere ein Button zum Öffnen des TMY-Konfigurationsdialogs. Gleichzeitig wird die manuelle Eingabe von Bewölkungswerten deaktiviert, um Konflikte mit den aus TMY bezogenen Strahlungswerten zu vermeiden. Wird die TMY-Nutzung deaktiviert, werden laufende Downloadprozesse kontrolliert abgebrochen und vorhandene TMY-Dateien aus dem Session-Verzeichnis entfernt.

6.1.2 Asynchroner Download mit Fehlerbehandlung und Debounce-Mechanismus

Der Download der TMY-Daten erfolgt nun vollständig asynchron. Änderungen an Eingabeparametern wie Latitude, Longitude oder dem definierten Stichprobenzeitraum lösen zeitverzögert (DOWNLOAD_DELAY) eine Anfrage an den PVGIS-Server aus. Dabei wird stets geprüft, ob bereits ein Download aktiv ist, der dann vor dem Start eines neuen Vorgangs abgebrochen wird. Mehrfache Anfragen mit identischer URI werden unterdrückt, um unnötige Netzwerklast zu vermeiden. Fehlerhafte oder unvollständige Downloads werden durch ein einfaches Dialogfenster gemeldet, sodass der Benutzer direkt auf Probleme aufmerksam gemacht wird.

6.1.3 TMY-Einstellungsfenster zur Visualisierung und Auswahl von Parametern

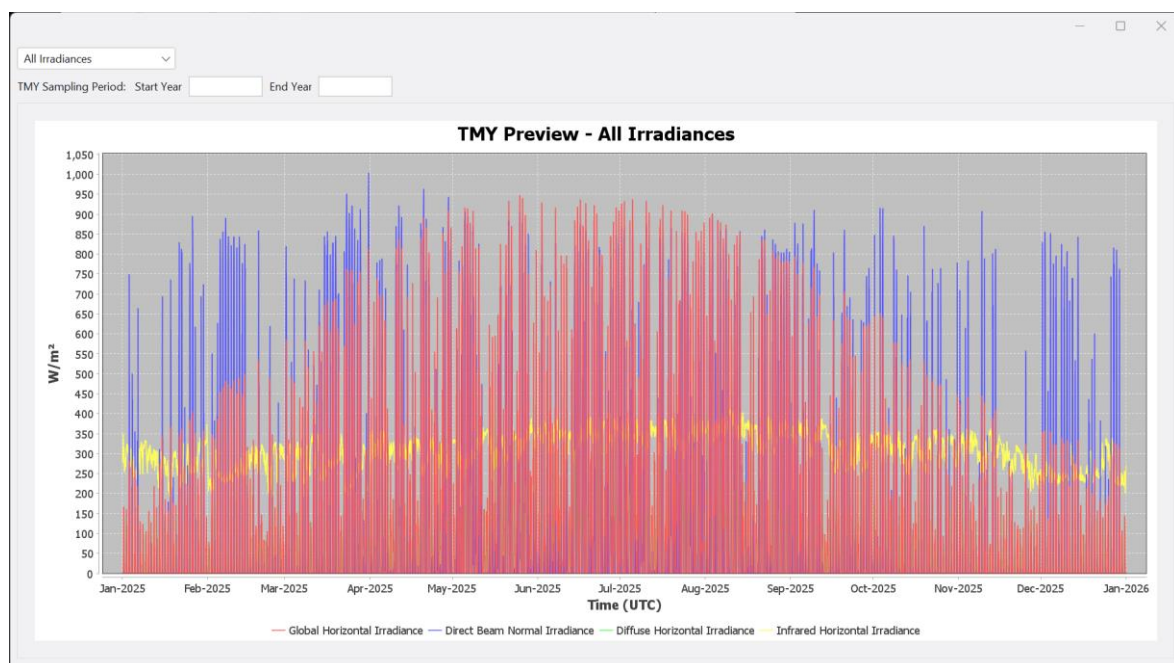


Abb. 15: TMY-Einstellungsfenster (eigene Abbildung)

Für die Analyse und Konfiguration der TMY-Daten wurde ein zusätzlicher Dialog (*TMYSettingPanel*) implementiert (Siehe Abb. 6). Er wird in einem separaten *JFrame* geöffnet und zeigt eine Vorschau der geladenen TMY-Daten an. Nutzer können den darzustellenden Messgrößentyp auswählen (Siehe Abb.7), darunter:

- globale, direkte und diffuse Strahlungsanteile
- Infrarotstrahlung
- Temperatur
- relative Luftfeuchte
- Windgeschwindigkeit und -richtung
- Luftdruck

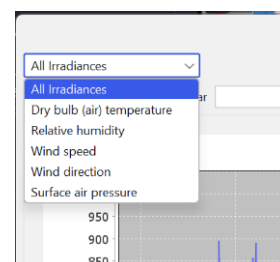


Abb. 16: Auswahl an Messgrößen für Vorschau (eigene Abbildung)

Die Start- und Endjahre der Stichprobe sind dabei direkt an das Domain-Modell gebunden. Änderungen an der Auswahl führen zur sofortigen Neuberechnung und -darstellung des Plots.

6.1.4 Visualisierung der TMY-Daten

Die visualisierten TMY-Zeitreihen basieren auf *JFreeChart* und nutzen eine einheitliche Zeitbasis. Da die PVGIS-Daten kein konsistentes Jahr besitzen, werden alle Zeitstempel automatisch auf das aktuelle Kalenderjahr normiert, um übersichtliche Jahresplots zu ermöglichen. Fehlerhafte Zeitstempel (durch Formatprobleme oder fehlende Werte) werden einzeln ignoriert, ohne die Gesamtvisualisierung zu beeinträchtigen. Dadurch bleibt die Plot-Darstellung robust gegenüber unvollständigen Datensätzen.

6.1.5 Integration in die Session-Verwaltung

Nach erfolgreichem Download wird die TMY-JSON-Datei im Session-Verzeichnis abgelegt und der Dateiname im *LocationAndTime*-Objekt registriert. Die Funktion *deleteTmy* erlaubt es, diese Datei wieder zu entfernen und den Zustand zurückzusetzen. Dadurch bleibt die TMY-Nutzung konsistent über mehrere Programmdurchläufe hinweg und ist klar vom Live-Downloadgeschehen getrennt.

6.1.6 Erweiterung der Spektrums-Vorschau

Die bestehende Spektrums-Vorschau wurde erweitert, um die neuen TMY-Daten sinnvoll einzubinden. Bei aktivierter Sweep-Darstellung wird der spektrale Bereich einer Simulation hervorgehoben, wobei direkte und diffuse Komponenten gesondert visualisiert werden. Dies ermöglicht eine unmittelbare Kontrolle darüber, wie sich meteorologische Eingangsgrößen auf die spektrale Verteilung auswirken. Die Legende sowie die Farbcodierung wurden an die erweiterten Anforderungen angepasst.

6.1.7 Zusammenfassung der Resultate

Die TMY-Integration bringt eine deutliche funktionale Erweiterung der SETFOS-Benutzeroberfläche:

- Neue TMY-Steuerung (Checkbox, Settings-Dialog)
- Asynchroner, optimierter Download mit Abbruchlogik und Fehlerfeedback
- Persistente Speicherung der TMY-JSON in der Sitzung
- Interaktive grafische Analyse meteorologischer Daten
- Einbettung in bestehende Spektrums-Vorschau

Insgesamt führt dies zu einer verbesserten Benutzerführung, erhöhter Simulationstransparenz und einer deutlich praxisnäheren Modellierung der Umweltbedingungen.

6.2 Kernel

Die Kernel-Erweiterungen ermöglichen die durchgängige Nutzung von TMY-Daten innerhalb der Simulation. Sie stellen sicher, dass meteorologische Zeitreihendaten konsistent verarbeitet, systemweit verfügbar gemacht und in die numerische Berechnung integriert werden, ohne bestehende Workflows zu verändern.

6.2.1 Neuer Kernel-Workflow für TMY-Daten

Im Kernel wurde ein klar strukturierter Workflow zur Verarbeitung von TMY-Daten implementiert. Dieser umfasst das Einlesen der TMY-Parameter, die zentrale Verwaltung meteorologischer Zeitreihen sowie deren zeitabhängige Nutzung in der Energieertragsberechnung. Die Aktivierung der TMY-Nutzung erfolgt konfigurationsgesteuert und bleibt optional.

6.2.2 Erweiterung der Parsing- und Parameterinfrastruktur

Die Parsing-Infrastruktur wurde um TMY-spezifische Parameter erweitert. Neue Tokens erlauben die Referenzierung von TMY-Dateien sowie die Definition des Auswertzeitraums. Die

Konfigurationsinformation wird konsistent über Parameterklassen und generische Setter im Kernel propagiert.

6.2.3 Zentrale Verarbeitung meteorologischer Zeitreihendaten

Mit dem TMYFileParser wurde eine dedizierte Kernel-Komponente zur Verarbeitung meteorologischer Daten eingeführt. Sie übernimmt das Einlesen, Validieren und zeitabhängige Bereitstellen der TMY-Daten und entkoppelt externe Dateiformate vollständig von der Simulationslogik.

6.2.4 TMY-basierte Energieertragsberechnung

Die Energieertragsberechnung wurde um eine zeitabhängige Skalierung der spektralen Intensitäten erweitert. Stündliche TMY-Daten fließen direkt in die Berechnung ein und ermöglichen eine realitätsnahe Modellierung variierender Umweltbedingungen, ohne bestehende Berechnungsmodelle zu verändern.

1.2.5 Zusammenfassung der Kernel-Resultate

Die Kernel-Erweiterungen führen zu einer robusten und modularen Integration von TMY-Daten:

- konfigurationsgesteuerter TMY-Workflow
- erweitertes Parsing und konsistente Parameterpropagierung
- zentrale Verwaltung meteorologischer Zeitreihen
- realitätsnahe, zeitaufgelöste Ertragsberechnung
- vollständige Rückwärtskompatibilität

Damit stellt der Kernel eine stabile und erweiterbare Grundlage für TMY-basierte Simulationen dar.

7 Diskussion und Ausblick

Im Folgenden werden die wichtigsten Erkenntnisse aus der Umsetzung der TMY-Integration zusammengefasst und kritisch eingeordnet. Zudem wird ein Ausblick auf mögliche Erweiterungen und Verbesserungen sowie organisatorische Lehren aus dem Projektverlauf gegeben.

7.1 Technische Bewertung der Umsetzung

Die Integration der TMY-Funktionalität in das bestehende SETFOS-Ökosystem erweitert das Simulationstool um einen essenziellen Baustein zur realitätsnahen Bewertung von Solarzellen. Die Kombination aus neuem TMY-Workflow, asynchronem Downloadmechanismus, Debounce-Strategie und einer eigenständigen Visualisierungskomponente (*TMYSettingPanel*) ermöglicht eine flexible und robuste Nutzung meteorologischer Datensätze.

Die praktische Umsetzung zeigte jedoch, dass insbesondere die harmonisierte Interaktion zwischen GUI, asynchronen Prozessen (*Executor Services*, *Futures*, *CompletableFutures*) und dem C++-Kernel unerwartet komplex war. Die Vielzahl an UI-Zuständen und Event-Abhängigkeiten erforderte zusätzlichen Aufwand und erhöhte die Testintensität.

7.2 Nutzerfreundlichkeit und Bedienbarkeit

Die GUI-Erweiterung liefert eine deutliche Verbesserung der Bedienbarkeit. Die klare Trennung zwischen TMY-Aktivierung, Einstellungen und Visualisierung ermöglicht eine intuitive Benutzerführung. Die Echtzeit-Reaktionen des Systems, insbesondere automatische Neuberechnungen bei Einstellungsänderungen, tragen zu einer höheren Transparenz bei.

Wahrnehmbare Einschränkungen bestehen jedoch im Performance-Verhalten bei großen TMY-Datensätzen. Das vollständige Neuaufbauen der Plots bei jeder Interaktion ist zwar robust, aber nicht

optimal hinsichtlich Ressourcenverbrauch. Eine inkrementelle Aktualisierung der Charts oder ein Caching-Mechanismus für bereits generierte Datensätze könnte hier langfristig zu Performancegewinnen führen.

7.3 Projektmanagement und gewonnene Erkenntnisse

Neben den technischen Ergebnissen lieferte das Projekt auch wertvolle Erkenntnisse hinsichtlich Planung, Umsetzung und Strukturierung von Entwicklungsarbeiten in einer bestehenden Softwarearchitektur. Insbesondere die Integration der TMY-Funktionalität zeigte, wie wichtig eine klare Aufteilung von Verantwortlichkeiten, eine schrittweise Implementierung sowie kontinuierliche Validierung sind. Der folgende Abschnitt fasst zentrale Erfahrungen und Lehren zusammen, die im Verlauf des Projekts gewonnen wurden und als Orientierung für zukünftige Erweiterungen und ähnliche Entwicklungsaufgaben dienen können.

7.3.1 Aufwand für GUI- und Kernel-Integration

In Planung zeigte sich, dass der erforderliche Aufwand für die Kopplung der neuen TMY-Funktionalitäten mit der bestehenden GUI-Struktur sowie der Integration in den Kernel unterschätzt wurde. Insbesondere folgende Aspekte spielten hierbei eine Rolle:

- **Unterschätzung des Umfangs bestehender Abhängigkeiten:** Die enge Verzahnung von Swing-GUI, Eventhandling, Datenbindung und Kernel-Aufrufen erwies sich komplexer als ursprünglich angenommen.
- **Mangelnde Pufferzeiten:** Für zentrale Integrationsschritte war kein ausreichender Zeitpuffer vorgesehen, was den Projektdruck in den späteren Phasen erhöhte.

Das Setup des Kernels insbesondere stellte sich als erheblich aufwändiger als der GUI dar, da der Kernel viele verschiedene Abhängigkeiten aus unterschiedlichen Quellen benötigte. Einige waren in speziellen Git Submodules einfach auffindbar, andere waren auf einem lokalen Fileshare-Server (welcher auch nur innerhalb des Fluxim-Netzwerkes erreichbar war) und wieder andere mussten manuell installiert werden. Dabei traten verschiedene zusätzliche Schwierigkeiten auf:

- Kompilierskripts für viele Abhängigkeiten konnten aufgrund des lokalen Fileshares nur vor Ort in den Räumlichkeiten der Fluxim ausgeführt werden.
- Das Ausführen von Kompilierskripts dauerte immer sehr lange und überschritt manchmal den verfügbaren Arbeitsspeicher der Entwicklungssysteme.
- Verschiedene Abhängigkeiten waren nicht oder nur teilweise dokumentiert, weshalb deren Notwendigkeit erst zu Tage kam, als die Kompilierskripts fehlschlagen.
- Der Kernel-Teil von SETFOS macht Gebrauch von implementations-abhängigen Funktionen der C++ Compiler (`#pragma[10]`), welche unter anderen Compilern zum Teil nicht funktionieren oder Abänderungen benötigten.
- Verschiedene Umgebungsvariablen waren nicht vorhanden und mussten mangels verfügbarer Dokumentation auf inkonsistente Arten eingespiessen werden.
- Das Entwicklungsteam hatte zusätzlich erheblich weniger Erfahrung mit C++ als mit Java.

Gesamthaft benötigte das Aufsetzen des Kernels etwa 2,5 Wochen, was den geplanten Zeitrahmen sprengte.

Diese Punkte verdeutlichen die Notwendigkeit eines strukturierten Projektmonitorings, klarer Aufwandsschätzungen sowie kontinuierlicher Risikoabschätzungen während der Implementierungsphase.

7.3.2 Planung und Arbeitsstruktur

Im Folgenden werden zentrale Erkenntnisse zur Planung und Arbeitsorganisation zusammengefasst, die während des Projektverlaufs sichtbar wurden und als Grundlage für zukünftige Verbesserungen dienen können.

- Regelmässige Meetings waren hilfreich, erforderten jedoch eine konsequentere Vorbereitung mit klaren Fragen und konkreten Diskussionspunkten.
- Ein detaillierter Gesamtplan mit frühzeitigem Überblick über technische Abhängigkeiten und Arbeitsschritte fehlte, wodurch das vorausschauende Arbeiten erschwert wurde.
- Mehr Eigenverantwortung im Projektverlauf hätte die Einbringung eigener Lösungsvorschläge und Ideen weiter fördern können.
- Ein höherer Grad an Eigenständigkeit bei der Einarbeitung in neue Themen und Werkzeuge erwies sich als notwendig.
- Das initiale Aufsetzen der Entwicklungsumgebung nahm deutlich mehr Zeit in Anspruch als erwartet, insbesondere durch die Arbeit auf zwei Betriebssystemen, was zu doppeltem Konfigurationsaufwand führte.
- Ein vergleichsweise grosser Anteil der Projektzeit wurde in architektonische Diskussionen investiert, während die eigentliche Implementierungsphase dadurch knapper ausfiel.

Eine strukturiertere Planung mit klar definierten Meilensteinen, Zuständigkeiten und einem realistischen Zeitrahmen würde zukünftige Projekte effizienter und planbarer gestalten.

7.4 Ausblick

Die aktuelle Implementierung bildet eine solide Grundlage für eine weiterführende Entwicklung der meteorologischen Integration in SETFOS. Zukünftige Erweiterungen könnten die nachfolgenden Aspekte adressieren:

- Performance-Optimierungen: Wiederverwendung bestehender Chart-Objekte, Hintergrund-Vorverarbeitung großer Datensätze und effizientere Speicherverwaltung.
- Erweiterte Visualisierungsfunktionen: Vergleich mehrerer TMY-Datensätze oder Overlays von Messdaten und Simulationsergebnissen.
- Verbesserte Fehlerrobustheit: Konsistente GUI-Reaktionen bei Netzwerkfehlern, Timeout-Strategien, Retry-Mechanismen.
- Verwendung weiterer Daten: Momentan werden nur Datenpunkte zur direkten und diffusen Sonnenstrahlung verwendet, jedoch sind auch Luftfeuchtigkeit, Windgeschwindigkeit und Lufttemperatur für die Berechnung relevant.

8 Verzeichnisse

A. Literaturverzeichnis

- [1] «Setfos: Simulation Software for OLEDs and Perovskite Solar Cells», Fluxim. Zugriffen: 1. November 2025. [Online]. Verfügbar unter: <https://www.fluxim.com/setfos-intro>
- [2] N. Armaroli und V. Balzani, «Solar Electricity and Solar Fuels: Status and Perspectives in the Context of the Energy Transition», *Chem. – Eur. J.*, Bd. 22, Nr. 1, S. 32–57, 2016, doi: 10.1002/chem.201503580.
- [3] F. Creutzig, P. Agoston, J. C. Goldschmidt, G. Luderer, G. Nemet, und R. C. Pietzcker, «The underestimated potential of solar energy to mitigate climate change», *Nat. Energy*, Bd. 2, Nr. 9, S. 17140, Aug. 2017, doi: 10.1038/nenergy.2017.140.
- [4] N. M. Haegel u. a., «Terawatt-scale photovoltaics: Transform global energy», *Science*, Bd. 364, Nr. 6443, S. 836–838, Mai 2019, doi: 10.1126/science.aaw1845.
- [5] Konrad Mertens, «Photovoltaik - Lehrbuch zu Grundlagen, Technologie und Praxis», 2022.
- [6] «PVGIS typical meteorological year (TMY) generator - The Joint Research Centre: EU Science Hub». Zugriffen: 1. November 2025. [Online]. Verfügbar unter: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/pvgis-tools/pvgis-typical-meteorological-year-tmy-generator_en
- [7] «API non-interactive service - The Joint Research Centre: EU Science Hub». Zugriffen: 1. November 2025. [Online]. Verfügbar unter: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/getting-started-pvgis/api-non-interactive-service_en
- [8] «PVGIS 5.3 - The Joint Research Centre: EU Science Hub». Zugriffen: 8. November 2025. [Online]. Verfügbar unter: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/pvgis-releases/pvgis-53_en
- [9] N. Aoun, «Methodology for predicting the PV module temperature based on actual and estimated weather data», *Energy Convers. Manag.* X, Bd. 14, S. 100182, Mai 2022, doi: 10.1016/j.ecmx.2022.100182.

B. Abbildungsverzeichnis

Abb. 1: Solarzelle und Solarmodule als Grundbausteine der Photovoltaik (K. Mertens, 2022, S.32)	2
Abb. 2: Typische Silizium-Solarzelle (K. Mertens, 2022, S.92).....	2
Abb. 3: Formel thermische Energiebilanz (N. Aoun, 2022, S.5)	4
Abb. 4: Formel der temperaturabhängige Modulwirkungsgrad (N. Aoun, 2022, S.5).....	4
Abb. 5: Formel gesamter Wärmeverluste(N. Aoun, 2022, S.5)	4
Abb. 6: Boden- und Himmelsstemperaturen (N. Aoun, 2022, S.5).....	5
Abb. 7: Formel Mean Bias Error (N. Aoun, 2022, S.5)	5
Abb. 8: Formel Mean Percentage Error (N. Aoun, 2022, S.5)	5
Abb. 9: Formel Root Mean Square Error (N. Aoun, 2022, S.6)	6
Abb. 10: Formel relative RMSE (N. Aoun, 2022, S.6)	6
Abb. 11: Systemablauf und Implementierungsworkflow (eigene Abbildung)	8
Abb. 12: GUI Workflow (eigene Abbildung).....	10
Abb. 13: Kernel Workflow (eigene Abbildung).....	15
Abb. 14: Erweiterung mit CheckBox Use TMY (eigene Abbildung)	24
Abb. 15: TMY-Einstellungsfenster (eigene Abbildung).....	25
Abb.16: Auswahl an Messgrößen für Vorschau (eigene Abbildung)	25

- Anhang

A. Anhang 1: Projektauftrag vom Complexis

Die erwartete Energieausbeute von Solarzellen hängt einerseits vom Zellschichtaufbau und dem Wirkungsgrad ab. Andererseits wird für die Berechnung der Energieausbeute das tageszeit- und ortsabhängige Sonnenspektrum benötigt. Die ZHAW Spin-off Firma Fluxim hat mit SETFOS eine Simulationssoftware entwickelt, welche diese Punkte berücksichtigt und weltweit in Forschungslabors eingesetzt wird. Aktuell fehlt jedoch die Verwendung von meteorologischen Daten, die Aussentemperatur, Wind sowie Bewölkung einbezieht. Hierfür wären ein typisches meteorologisches Jahr (Typical Meteorological Year, TMY) an einem beliebigen Ort verwendbar und mit den bereits in SETFOS implementierten physikalischen Modellen zu verbinden. Dazu bietet sich ein TMY Datenimport und/oder eine Schnittstelle an einen TMY Generator (Bsp. PVGIS Software) an. Damit würde die mit Setfos bereits mögliche Berechnung des jährlichen Energieertrags (Energy Yield, EY) von benutzerdefinierten Solarzellen verbessern. Das Projekt wird von Informatikern und Physikern des Industriepartners Fluxim AG begleitet und hat einen Anteil Java (für das GUI) und C++ (Simulationskern).

Die Hauptaufgaben in diesem Projekt umfassen:

- Einarbeitung in die Methode zur Berechnung von Energieertrag von Solarzellen sowie in Formate für TMY Datensätze von verschiedenen Anbietern, inkl. der API Schnittstelle der PVGIS Software
- Erweiterung der grafischen Oberfläche von SETFOS: a) zur Einbindung der API Schnittstelle, b) zur interaktiven Ortswahl und für Generation, Download und Verwaltung von TMY Daten, c) Schnittstelle zur Übermittlung der TMY Daten vom User Interface zum Berechnungskern
- Erweiterung des Berechnungskerns für die Integration der Solarzellenleistung über eine benutzerdefinierte Zeitperiode mit Verrechnung der TMY Daten.
- Tests der Berechnung, Evaluation der Rechenzeit, allfällige Beschleunigungsmassnahmen
- Allfälliges Benchmarking zwischen rein optischer und voller opto-elektronischer Simulationen, bei denen die Strom-Spannungs-Kennlinie der Solarzellen nicht approximiert sondern berechnet wird.

Die Arbeit wird in Zusammenarbeit mit Softwareentwicklern und Physikern des Industriepartners Fluxim AG ausgeführt, welche die SETFOS Simulationssoftware entwickeln und international kommerzialisieren. Fluxim ist eine Spin-off-Firma des Institute of Computational Physics der ZHAW in Winterthur und hat sich inzwischen als ein führender Anbieter von wissenschaftlicher Simulationssoftware und Messsystemen für die globale Forschung an neuartigen Solarzellen und Bildschirmtechnologien etabliert.

Voraussetzungen

- Kenntnisse in C++, Java und API Schnittstellen sind von Vorteil
- Interesse an Solarenergie